Algorithms: Design and Analysis (Part I)

Mihir Rao

Contents

1 Introduction: Integer Multiplication and Merge Sort **4**
1.1 Integer multiplication and the cost of a step . . . 4
1.2 Recursive multiplication and Karatsuba's idea . . . 4
1.3 Divide and conquer through sorting . . . 5
1.4 Merging two sorted arrays . . . 6
1.5 Counting work in one merge . . . 7
1.6 The recursion-tree analysis . . . 7
1.7 Three principles for algorithm analysis . . . 8

2 Asymptotic Analysis **10**
2.1 Counting loops before simplifying . . . 10
2.2 Big-O: an eventual upper bound . . . 11
2.3 Polynomial examples and fixed witnesses . . . 12
2.4 Omega and Theta: lower and matching bounds . . . 12
2.5 Little-o and the order of the quantifiers . . . 14
2.6 Constants inside an exponent . . . 14
2.7 A sum and a maximum have the same growth . . . 15

3 Divide and Conquer Algorithms **16**
3.1 Counting inversions . . . 16
3.2 Piggybacking on merge sort . . . 17
3.3 Matrix multiplication and Strassen's algorithm . . . 18
3.4 The closest pair problem . . . 19
3.5 Correctness of the split-pair search . . . 20

4 The Master Method **23**
4.1 Recurrences for integer multiplication . . . 23
4.2 The precise statement . . . 23
4.3 Examples . . . 24
4.4 The recursion tree . . . 24
4.5 Intuition for the three cases . . . 25
4.6 Completing the proof . . . 25

5 QuickSort: Algorithm **27**
5.1 Partitioning around a pivot . . . 27
5.2 In-place partitioning . . . 28
5.3 Correctness . . . 29
5.4 The importance of the pivot . . . 30
5.5 Random pivots . . . 30

6 QuickSort: Analysis **31**
6.1 Reducing running time to comparisons . . . 31
6.2 Decomposing the comparison count . . . 31

6.3 The key insight . . . 32
6.4 Final calculation . . . 33

7 Probability Review **35**
7.1 Sample spaces and events . . . 35
7.2 Random variables and expectation . . . 35
7.3 Linearity of expectation . . . 36
7.4 Conditional probability . . . 36
7.5 Independence . . . 37

8 Linear-Time Selection **39**
8.1 The selection problem . . . 39
8.2 Partitioning and randomized selection . . . 39
8.3 Analysis of randomized selection . . . 40
8.4 Deterministic selection . . . 41
8.5 Analysis of deterministic selection . . . 42
8.6 A sorting lower bound . . . 43

9 Graphs and the Contraction Algorithm **45**
9.1 Graphs and cuts . . . 45
9.2 Representing graphs . . . 46
9.3 The random contraction algorithm . . . 46
9.4 Success probability of one run . . . 47
9.5 Repeated trials . . . 48
9.6 Counting minimum cuts . . . 49

10 Graph Search and Connectivity **50**
10.1 Generic graph search . . . 50
10.2 Breadth-first search . . . 51
10.3 Shortest paths via BFS . . . 52
10.4 Undirected connectivity . . . 52
10.5 Depth-first search . . . 53
10.6 Topological sort . . . 53
10.7 Strongly connected components . . . 55
10.8 Correctness of Kosaraju's algorithm . . . 56
10.9 Structure of the Web . . . 57

11 Dijkstra's Shortest-Path Algorithm **59**
11.1 Single-source shortest paths . . . 59
11.2 The algorithm . . . 59
11.3 Negative edge lengths . . . 60
11.4 Correctness . . . 61
11.5 A heap-based implementation . . . 62

12 Heaps **64**
12.1 Heap operations . . . 64
12.2 Applications . . . 64
12.3 The heap property . . . 65
12.4 Insert and bubble-up . . . 66
12.5 Extract-Min and bubble-down . . . 67

13 Balanced Binary Search Trees **69**
13.1 Sorted arrays and balanced search trees . . . 69
13.2 Binary search tree structure . . . 69
13.3 Search, insert, min, max, and predecessor . . . 70

13.4 In-order traversal and deletion . . . 71
13.5 Select and rank . . . 71
13.6 Red-black trees . . . 72
13.7 Rotations . . . 73
13.8 Insertion into a red-black tree . . . 74

14 Hashing: The Basics **76**
14.1 Applications . . . 76
14.2 Buckets and hash functions . . . 77
14.3 Resolving collisions . . . 78
14.4 What makes a good hash function . . . 78

15 Universal Hashing **80**
15.1 The load of a hash table . . . 80
15.2 Pathological data sets . . . 80
15.3 Universal families of hash functions . . . 81
15.4 A universal family for IP addresses . . . 81
15.5 Chaining with a universal family . . . 82
15.6 Open addressing: heuristic analysis . . . 83

16 Bloom Filters **84**
16.1 The data structure . . . 84
16.2 Heuristic analysis . . . 85

Lecture 1: Introduction: Integer Multiplication and Merge Sort

An algorithm turns an input into a required output. Design chooses the steps; analysis establishes correctness and cost.

Two familiar tasks, multiplying integers and sorting numbers, introduce a recurring question: can we improve on the obvious method?

1.1 Integer multiplication and the cost of a step

The input is two nonnegative integers x and y, each represented with n decimal digits; the required output is xy. Here a primitive operation adds or multiplies two single digits. Multiplying two arbitrarily long integers is therefore not one primitive operation.

The grade-school method forms one partial product for each digit of y, then adds the shifted rows. For the given example,

$$\begin{array}{r} 5678 \\ \times 1234 \\ \hline 22712 \\ 170340 \\ 1135600 \\ 5678000 \\ \hline 7006652 \end{array}$$

There are n rows, each involving work proportional to n, including digit multiplication and carrying. The final additions also take at most a constant times n^2 work. Thus the total grows quadratically with the number of digits. Doubling the digit count roughly quadruples this work.

1.2 Recursive multiplication and Karatsuba's idea

Split each input into a high half and a low half. With $m = n/2$ for an even digit count, write

$$x = 10^m a + b, \qquad y = 10^m c + d,$$

where each block uses at most m digits, allowing leading zeros. Expanding the product gives

$$xy = 10^{2m} ac + 10^m (ad + bc) + bd. \tag{1.1}$$

This identity reduces the original task to products of smaller integers. A direct recursive method computes ac, ad, bc, bd separately, then combines them using equation (1.1). Small inputs are

handled directly.

Karatsuba multiplication reuses information to remove one of these four recursive products. Compute

$$u = ac, \qquad v = bd, \qquad w = (a+b)(c+d).$$

Since $w = ac + ad + bc + bd$, the middle coefficient is $w - u - v = ad + bc$. Therefore

$$\boxed{xy = 10^{2m}u + 10^{m}(w - u - v) + v.}$$

There are three recursive multiplications, plus additions, subtractions, and decimal shifts. Sums such as $a + b$ can have an extra carry digit; a complete implementation must preserve it. The smaller number of recursive products is the reason to investigate this method further, rather than counting a recursive call as a single inexpensive step.

Example: For $x = 5678$ and $y = 1234$, take $a = 56$, $b = 78$, $c = 12$, $d = 34$. Then

$$u = 56 \cdot 12 = 672, \qquad v = 78 \cdot 34 = 2652,$$
$$w = 134 \cdot 46 = 6164, \quad w - u - v = 2840.$$

Recombining gives

$$10^4(672) + 10^2(2840) + 2652 = 7006652,$$

which agrees with the grade-school calculation.

1.3 Divide and conquer through sorting

A divide and conquer algorithm splits a task into smaller subproblems, solves them recursively, and combines their answers. The combination step matters: reducing the input size is useful only if the work of solving and combining the subproblems is controlled.

Definition 1.1 [Sorting problem]. Given an array of n distinct numbers, return the same numbers in increasing order. Correctness requires both ordering and preservation of the input elements.

Merge sort splits the array into two halves, sorts each half recursively, and merges the sorted halves. Arrays of length zero or one are already sorted.

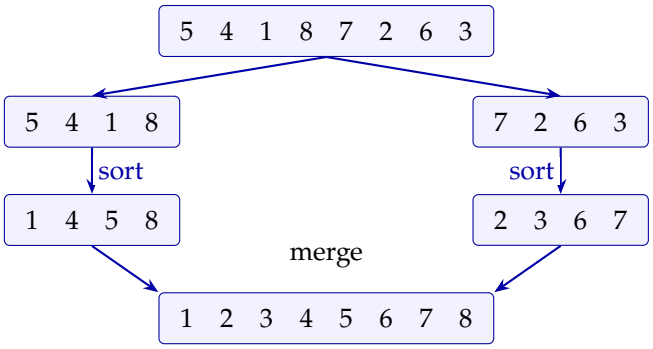


Figure 1: Merge sort on the supplied eight-element array. The two recursive calls return sorted halves; the final merge combines them in increasing order.

Algorithm 1.1: Merge sort

Input: Array X of length n, with distinct entries.
Output: The entries of X in increasing order.

```
if n ≤ 1 then
    return X
m ← ⌊n/2⌋
A ← MergeSort(X[1 : m])
B ← MergeSort(X[m + 1 : n])
return Merge(A, B)
```

Both recursive arrays have length smaller than n when $n \geq 2$, so the procedure eventually reaches a base case. The floor handles odd lengths without discarding an element.

1.4 Merging two sorted arrays

The smallest unused element of a sorted array is its first unused element. To find the next output element, it is therefore enough to compare the two current front elements. After choosing one, advance only its pointer. If an array is exhausted, copy the remaining elements of the other array.

Algorithm 1.2: Merge two sorted arrays

Input: Increasing arrays $A[1 : p]$ and $B[1 : q]$, with distinct entries overall.
Output: An increasing array C containing every input entry once.

```
i ← 1; j ← 1; C ← an empty array
while i ≤ p and j ≤ q do
    if A[i] < B[j] then
        Append A[i] to C; i ← i + 1
    else
        Append B[j] to C; j ← j + 1
Append the remaining suffix A[i : p], if nonempty, to C
Append the remaining suffix B[j : q], if nonempty, to C
return C
```

Lemma 1.2 [Correctness of merging]. *If A and B are sorted, the merge procedure returns their entries in increasing order, with no loss or repetition.*

Proof. The idea is to maintain that the output contains exactly the smallest entries processed so far. Initially it is empty. While both arrays have unused entries, the smaller front element is the smallest unused element of their union, because neither sorted suffix contains a smaller element behind its front. Appending that element preserves the ordering and moves exactly one pointer. When one array is exhausted, the remaining sorted suffix contains all remaining elements, each at least as large as the last element appended. Copying it finishes the output. Every element is copied once, so the result has the required order and contents. □

Proposition 1.3 [Correctness of merge sort]. *Merge sort returns the input entries in increasing order.*

Proof. Use induction on the array length. Lengths zero and one are base cases. For $n \geq 2$, each half is shorter than the input, so the induction hypothesis guarantees that the recursive calls sort both halves correctly. The merge lemma then orders their union and preserves all entries. This proves the claim for length n. □

1.5 Counting work in one merge

Let $m = p + q$ be the total number of input entries. Each iteration copies one entry, and the suffix copies account for exactly the remaining entries. Thus merging takes work proportional to m.

For a coarse operation count, use the simplified loop accounting: two pointer initializations and at most four elementary steps for each output position. This gives

$$\text{MergeCost}(m) \leq 4m + 2 \leq 6m \qquad (m \geq 1).$$

The constant depends on which bookkeeping steps are charged. Completing the end cases in actual code can change it, but does not change the linear growth. This bound is a mathematical accounting model, not a count of machine instructions.

1.6 The recursion-tree analysis

Each merge is linear in its own input size, but there are many merges. The useful observation is that the total input size across any one level of recursion is n.

Assume first that $n = 2^h$, so all splits are even. A recursion tree has one node per recursive call. At depth j there are 2^j subproblems, each of size $n/2^j$. The leaves have size one, so they occur at depth $h = \log_2 n$. There are $h + 1$ levels, counting the root and the leaves.

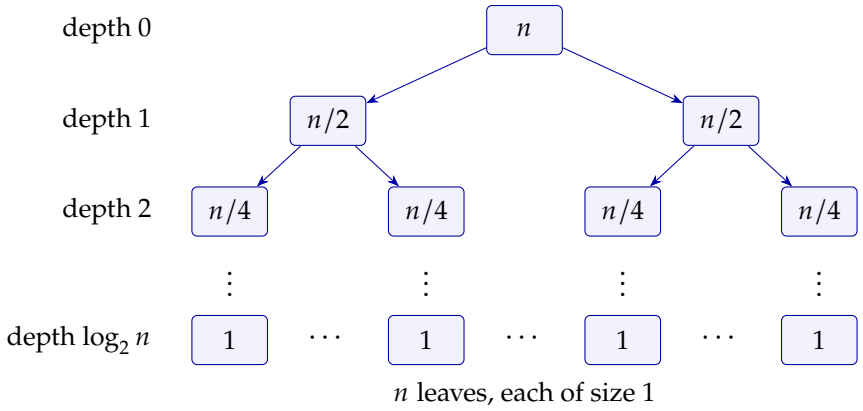


Figure 2: Schematic recursion tree for n a power of two. The number of subproblems doubles as their sizes halve. Intermediate levels are omitted.

Theorem 1.4 [Merge-sort running time]. *In the coarse operation model above, merge sort on $n = 2^h \geq 1$ entries uses at most $6n \log_2 n + 6n$ charged operations, including a constant charge for each leaf.*

Proof. Bound the work one level at a time. Charge at most $6m$ for a node of size m, with leaf work bounded by 6. The charge at depth j is at most

$$\underbrace{2^j}_{\text{number of nodes}} \quad \underbrace{6\left(\frac{n}{2^j}\right)}_{\text{work per node}} = 6n.$$

There are $\log_2 n + 1$ levels. Summing their charges gives

$$\sum_{j=0}^{\log_2 n} 6n = 6n(\log_2 n + 1) = 6n \log_2 n + 6n.$$

The leaf charge is included, even though a leaf performs no merge. □

For a general positive n, balanced splitting has at most $\lceil \log_2 n \rceil + 1$ levels. Nodes at any fixed depth cover disjoint portions of the input, whose total size is at most n. The same reasoning gives a bound proportional to $n \log n$. The power-of-two assumption makes the displayed level counts exact; it is not required by the algorithm.

1.7 Three principles for algorithm analysis

1. Worst-case analysis gives a running-time bound for every input of a given size. It is useful for a general-purpose routine whose future inputs are unknown. An average-case guarantee needs a model for which inputs are likely; a benchmark measures selected instances.

2. Suppress constant factors and lower-order terms when comparing growth rates. Constants

depend on implementation and computing hardware; retaining all of them can obscure the main design difference.

3. Asymptotic analysis studies how the cost grows as the input becomes large. A smaller growth rate need not give a smaller cost on every small input.

For example, compare the illustrative costs

$$f(n) = 6n \log_2 n + 6n, \qquad g(n) = \tfrac{1}{2}n^2.$$

The linearithmic expression eventually becomes much smaller, although the quadratic expression is smaller over part of the range of small n. These are cost formulas, not measured runtimes.

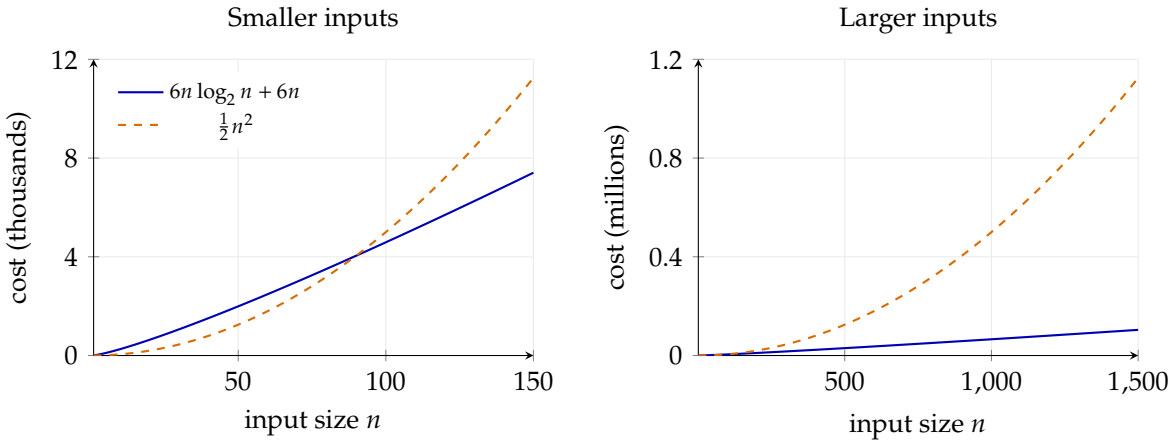


Figure 3: The supplied illustrative cost functions at two input scales. The relative advantage of $6n \log_2 n + 6n$ grows with the input size. The vertical axes use different units.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Introduction.

Lecture 2: Asymptotic Analysis

Asymptotic analysis compares operation counts as input size grows, suppressing constant factors and lower-order terms.

For example, a bound such as $6n \log_2 n + 6$ has the growth rate $n \log n$. The notation $O(n \log n)$ makes this comparison precise. Here n denotes the input size, such as the length of an array, and $T(n)$ usually denotes the worst-case running time over inputs of that size.

2.1 Counting loops before simplifying

Count each array comparison and loop update as constant work. First identify how many times that work is performed. A return statement can end an algorithm early, but a worst-case bound must also cover inputs on which no early return occurs.

Algorithm 2.1: Search one array

Input: Array $A[1 : n]$ and target t.
Output: Whether A contains t.

```
for i ← 1 to n do
    if A[i] = t then
        return true
return false
```

If t is absent, all n entries are compared with it. There is constant work per iteration, so the worst-case running time grows linearly with n. Finding t at the first position is a faster input, but does not reduce the worst-case bound.

Algorithm 2.2: Search two arrays in succession

Input: Arrays $A[1 : n]$, $B[1 : n]$, and target t.
Output: Whether either array contains t.

```
for i ← 1 to n do
    if A[i] = t then
        return true
for i ← 1 to n do
    if B[i] = t then
        return true
return false
```

When t is absent from both arrays, there are $n + n = 2n$ comparisons. The loops run in succession, so their work adds. Suppressing the factor 2 leaves the same linear growth rate as the first search.

Algorithm 2.3: Check two arrays for a common entry

```
Input: Arrays A[1 : n] and B[1 : n].
Output: Whether the arrays share an entry.
for i ← 1 to n do
    for j ← 1 to n do
        if A[i] = B[j] then
            return true
return false
```

If the arrays have no common entry, each of the n outer iterations executes all n inner iterations. There are n^2 comparisons, giving quadratic worst-case growth.

Algorithm 2.4: Check an array for duplicates

```
Input: Array A[1 : n].
Output: Whether two different positions have the same entry.
for i ← 1 to n do
    for j ← i + 1 to n do
        if A[i] = A[j] then
            return true
return false
```

Starting the inner loop at $i + 1$ avoids comparing an entry with itself or checking a pair twice. If all entries are distinct, the comparison count is

$$\sum_{i=1}^{n}(n-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

The leading term is $n^2/2$, so the growth is still quadratic. A changing inner-loop length requires summing the iteration counts; it does not by itself imply a faster asymptotic bound.

2.2 Big-O: an eventual upper bound

Let $T(n) \geq 0$ be a cost function, and let $f(n) > 0$ for sufficiently large positive integers n. We want to express that T eventually stays below a fixed multiple of f, even if the two functions behave differently on small inputs.

Definition 2.1 [Big-O]. Write $T(n) = O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 1$ such that

$$0 \leq T(n) \leq cf(n) \qquad \text{for every integer } n \geq n_0.$$

This is an asymptotic upper bound.

The fixed multiplier c absorbs constant factors, and n_0 excludes small inputs. Neither depends on n; the bound holds for every integer $n \geq n_0$.

An upper bound need not be tight: a linear running time is also $O(n^2)$. When comparing the loop examples, $O(n)$ and $O(n^2)$ describe their respective linear and quadratic growth, rather than the only upper bounds they satisfy.

2.3 Polynomial examples and fixed witnesses

Proposition 2.2 [Polynomial upper bound]. *For fixed real coefficients $a_0, \dots, a_k$ and an integer $k \geq 0$, let $p(n) = a_k n^k + \cdots + a_1 n + a_0$. Then $|p(n)| = O(n^k)$.*

Proof. The idea is to bound every lower power by the highest power. For $n \geq 1$ and $0 \leq i \leq k$, we have $n^i \leq n^k$. Hence

$$|p(n)| \leq \sum_{i=0}^{k} |a_i| n^i \leq \left(\sum_{i=0}^{k} |a_i| \right) n^k.$$

Choose $n_0 = 1$ and $c = \max\{1, \sum_{i=0}^{k} |a_i|\}$. These are fixed witnesses for the big-O definition. In particular, the bound applies to a nonnegative polynomial running time. □

Proposition 2.3 [Different polynomial degrees]. *For every integer $k \geq 1$, n^k is not $O(n^{k-1})$.*

Proof. The idea is that one uncancelled factor of n eventually exceeds every fixed constant. If $n^k = O(n^{k-1})$, there would be constants $c, n_0 > 0$ with $n^k \leq c n^{k-1}$ for all integers $n \geq n_0$. Dividing by the positive quantity n^{k-1} gives $n \leq c$ for all such n. Choose an integer $n > \max\{n_0, c\}$. This contradicts the required inequality. □

Prove an upper bound with fixed witnesses; disprove one by defeating every proposed pair on a sufficiently large input.

2.4 Omega and Theta: lower and matching bounds

Big-O bounds a function from above. A corresponding lower bound says that the cost eventually remains at least a fixed multiple of another function.

Definition 2.4 [Big-Omega]. Write $T(n) = \Omega(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 1$ such that

$$T(n) \geq c f(n) \qquad \text{for every integer } n \geq n_0.$$

This is an asymptotic lower bound.

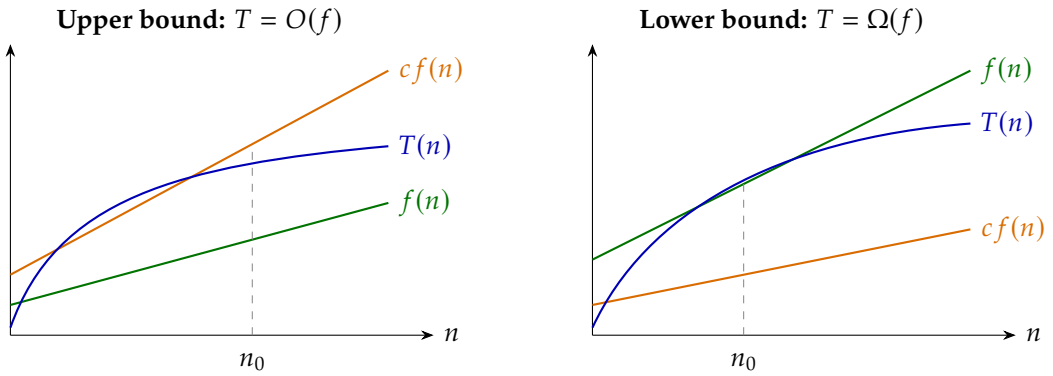


Figure 4: Schematic eventual bounds. For big-O, T lies below cf after n_0; for big-Omega, it lies above cf. The unscaled function f need not lie on the same side of T as its scaled version.

Definition 2.5 [Big-Theta]. Write $T(n) = \Theta(f(n))$ if both $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$. Equivalently, there exist constants $c_1, c_2 > 0$ and $n_0 \geq 1$ such that

$$c_1 f(n) \leq T(n) \leq c_2 f(n) \qquad \text{for every integer } n \geq n_0.$$

This is an asymptotically tight bound.

To obtain one threshold from separate upper and lower bounds, take the larger of their two thresholds. Then both inequalities hold simultaneously. Conversely, the two-sided inequality immediately supplies both bounds.

These symbols describe inequalities between functions. They do not select which inputs are considered: if $T(n)$ is a worst-case running time, then $\Omega(f(n))$ is a lower bound on that same worst-case function.

Example: Consider $T(n) = \frac{1}{2}n^2 + 3n$. For every integer $n \geq 1$,

$$\tfrac{1}{2}n^2 \leq T(n) \leq \tfrac{1}{2}n^2 + 3n^2 \leq 4n^2.$$

Thus $T(n) = \Theta(n^2)$, with $c_1 = 1/2$, $c_2 = 4$, and $n_0 = 1$. The other comparisons follow directly:

Claim	True?	Reason for $n \geq 1$
$T(n) = O(n)$	No	$T(n)/n = \frac{1}{2}n + 3$ is unbounded.
$T(n) = \Omega(n)$	Yes	$T(n) \geq \frac{1}{2}n^2 \geq \frac{1}{2}n$.
$T(n) = \Theta(n^2)$	Yes	$\frac{1}{2}n^2 \leq T(n) \leq 4n^2$.
$T(n) = O(n^3)$	Yes	$T(n) \leq 4n^2 \leq 4n^3$.

The cubic upper bound is valid, but the quadratic Theta bound describes the growth more precisely. Similarly, the four loop examples have tight worst-case bounds $\Theta(n)$, $\Theta(n)$, $\Theta(n^2)$, and $\Theta(n^2)$, respectively.

2.5 Little-o and the order of the quantifiers

A fixed upper multiplier allows two functions to grow at the same rate. To say that T eventually becomes smaller than *every* positive multiple of f, use a stronger comparison.

Definition 2.6 [Little-o]. Write $T(n) = o(f(n))$ if for every constant $c > 0$, there exists a threshold $n_0 \geq 1$ such that

$$0 \leq T(n) \leq cf(n) \qquad \text{for every integer } n \geq n_0.$$

For big-O, one suitable multiplier is enough. For little-o, the inequality must eventually work no matter how small the positive multiplier is. Here n_0 may depend on the chosen c, but not on the running variable n.

Proposition 2.7. *For every integer $k \geq 1$, $n^{k-1} = o(n^k)$.*

Proof. The idea is to make the ratio $1/n$ smaller than any given constant. Fix $c > 0$, and choose $n_0 = \max\{1, \lceil 1/c \rceil\}$. Then every integer $n \geq n_0$ satisfies $1/n \leq c$, so

$$n^{k-1} \leq cn^k.$$

Since this works for every positive c, the little-o definition holds. □

2.6 Constants inside an exponent

A constant added to an exponent behaves differently from a constant multiplied by the input size in that exponent.

Proposition 2.8. *$2^{n+10} = O(2^n)$, but 2^{10n} is not $O(2^n)$.*

Proof. Compare each expression with 2^n by division. First,

$$2^{n+10} = 2^{10}2^n = 1024 \cdot 2^n,$$

so $c = 1024$ and $n_0 = 1$ give the required upper bound.
If $2^{10n} = O(2^n)$, there would be fixed constants $c, n_0 > 0$ with $2^{10n} \leq c2^n$ for all integers $n \geq n_0$. Dividing by 2^n yields $2^{9n} \leq c$. Choose an integer $n > \max\{n_0, (\log_2 c)/9\}$. Then $2^{9n} > c$, a contradiction. □

The factor 10 in $10n$ cannot be discarded: after division, it leaves a ratio that grows without bound. The fixed additive shift +10 leaves only a constant ratio.

2.7 A sum and a maximum have the same growth

The larger of two nonnegative quantities controls their sum up to a factor of two. The identity is useful even when the two functions cross repeatedly, so neither is always the larger one.

Proposition 2.9 [Maximum and sum]. *For positive functions f and g on the positive integers,*

$$\max\{f(n), g(n)\} = \Theta(f(n) + g(n)).$$

Proof. Use pointwise inequalities, so no assumption about which function grows faster is needed. Each function is at most the maximum, and both are nonnegative. Therefore, for every $n \geq 1$,

$$\max\{f(n), g(n)\} \leq f(n) + g(n) \leq 2\max\{f(n), g(n)\}.$$

Rearranging gives

$$\tfrac{1}{2}\big(f(n) + g(n)\big) \leq \max\{f(n), g(n)\} \leq f(n) + g(n).$$

These are the Theta inequalities with $c_1 = 1/2$, $c_2 = 1$, and $n_0 = 1$. □

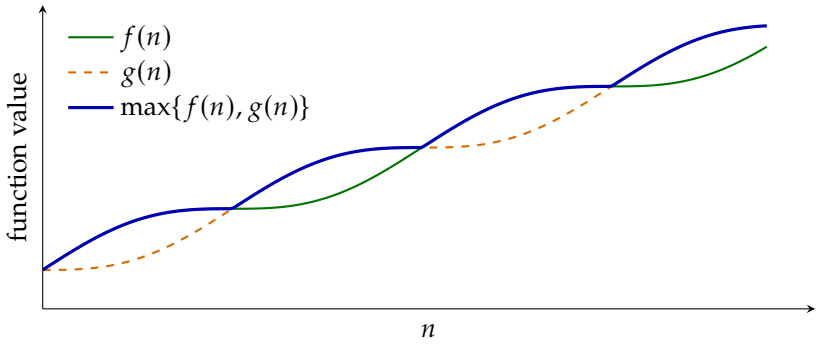


Figure 5: Schematic crossing functions. Their pointwise maximum follows whichever curve is higher. Their sum remains between this maximum and twice the maximum, including where the curves switch order.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Asymptotic Analysis.

Lecture 3: Divide and Conquer Algorithms

The divide and conquer paradigm has three steps: *divide* the input into smaller subproblems, *conquer* the subproblems recursively, and *combine* their solutions into one for the original problem. Three applications show that the combine step is usually where the cleverness lies.

3.1 Counting inversions

Definition 3.1 [Inversion]. Let $A[1:n]$ contain the numbers $1, 2, \ldots, n$ in some order. An inversion is a pair of indices (i, j) with $i < j$ and $A[i] > A[j]$. The task is to count the inversions of A.

For $A = (1, 3, 5, 2, 4, 6)$ the inversions, written as pairs of values, are $(3, 2)$, $(5, 2)$, and $(5, 4)$. Connect each value in sorted order to its position in A: two segments cross exactly when their values form an inversion. The count is a numerical measure of how similar two rankings are, which is useful, for example, in collaborative filtering.

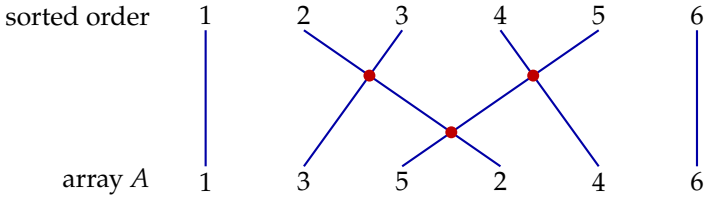


Figure 6: The inversions of $A = (1, 3, 5, 2, 4, 6)$ as crossings between sorted order and array order. The three marked crossings are the pairs $(3, 2)$, $(5, 2)$, and $(5, 4)$.

There are $\binom{n}{2} = n(n-1)/2$ pairs of indices, and the reversed array makes every pair an inversion. So a 6-element array has at most 15 inversions, and checking every pair takes $\Theta(n^2)$ time.

To do better, classify an inversion (i, j) with $i < j$ as

- left if $i, j \le n/2$ and right if $i, j > n/2$; these are inversions of one half and can be counted recursively;
- split if $i \le n/2 < j$; these need a separate subroutine.

If split inversions can be counted in $O(n)$ time, the recursion has the shape of merge sort, $T(n) \le 2T(n/2) + O(n)$, and runs in $O(n \log n)$ time.

3.2 Piggybacking on merge sort

The second key idea is to make each recursive call both count the inversions of its subarray *and* sort it. The merge step then uncovers the split inversions.

If A had no split inversions, every element of the sorted left half B would be smaller than every element of the sorted right half C, and merging would copy all of B before any of C. In general, consider merging $B = (1, 3, 5)$ and $C = (2, 4, 6)$. When 2 is copied to the output, the elements 3 and 5 remain in B; these give the split inversions $(3, 2)$ and $(5, 2)$. When 4 is copied, 5 remains, giving $(5, 4)$.

Lemma 3.2 [Split inversions during merging]. *Let B and C be the sorted first and second halves of A. The split inversions involving an element y of C are exactly the pairs (x, y) with x still remaining in B when y is copied to the output.*

Proof. Let x be an element of B. Since x lies in the first half and y in the second, (x, y) is a split inversion exactly when $x > y$. Merging always copies the smaller front element. If x is copied before y, then $x < y$, so they form no inversion. If y is copied while x remains in B, then $y < x$, so they form a split inversion. □

So the merge keeps a running total: whenever an element of C is copied, add the number of elements remaining in B. The extra work is $O(1)$ per copied element.

Algorithm 3.1: Sort and count inversions

Input: Array $A[1 : n]$ of distinct numbers.
Output: The sorted array and the number of inversions of A.

```
if n ≤ 1 then
    return (A, 0)
m ← ⌊n/2⌋
(B, X) ← SortAndCount(A[1 : m])
(C, Y) ← SortAndCount(A[m + 1 : n])
(D, Z) ← MergeAndCountSplitInv(B, C)
return (D, X + Y + Z)
```

Algorithm 3.2: Merge and count split inversions

Input: Sorted arrays $B[1 : p]$ and $C[1 : q]$.
Output: Their merge D and the number Z of pairs (x, y), $x \in B$, $y \in C$, with $x > y$.

```
i ← 1; j ← 1; Z ← 0; D ← an empty array
for k ← 1 to p + q do
    if j > q or (i ≤ p and B[i] < C[j]) then
        Append B[i] to D; i ← i + 1
    else
        Append C[j] to D; Z ← Z + (p − i + 1); j ← j + 1
return (D, Z)
```

Here $p - i + 1$ is the number of elements left in B, so by lemma 3.2, Z is the number of split inversions. Merging and maintaining the total each take $O(n)$ time, so SortAndCount satisfies the merge-sort recurrence and runs in $O(n \log n)$ time.

3.3 Matrix multiplication and Strassen's algorithm

For $n \times n$ matrices X and Y, the product $Z = XY$ has entries

$$z_{ij} = (\text{row } i \text{ of } X) \cdot (\text{column } j \text{ of } Y) = \sum_{k=1}^{n} X_{ik} Y_{kj}.$$

The input size is $\Theta(n^2)$. The straightforward iterative algorithm computes n^2 entries, each in $\Theta(n)$ time, for $\Theta(n^3)$ in total. For $n = 2$,

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{pmatrix}.$$

The same formula holds for blocks. Assume n is a power of 2 and write

$$X = \begin{pmatrix} A & B \\ C & D \end{pmatrix}, \qquad Y = \begin{pmatrix} E & F \\ G & H \end{pmatrix}, \qquad XY = \begin{pmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{pmatrix},$$

where A through H are $n/2 \times n/2$ matrices. The first recursive algorithm computes the eight block products recursively, then adds them in $\Theta(n^2)$ time. Its running time is still $\Theta(n^3)$, which follows from the master method of Lecture 4.

Strassen's algorithm (1969) recursively computes only seven cleverly chosen products,

$$\begin{aligned} P_1 &= A(F - H), & P_2 &= (A + B)H, & P_3 &= (C + D)E, & P_4 &= D(G - E), \\ P_5 &= (A + D)(E + H), & P_6 &= (B - D)(G + H), & P_7 &= (A - C)(E + F), \end{aligned}$$

and recovers the product with $\Theta(n^2)$ additions and subtractions. Replacing eight recursive products with seven makes the running time better than cubic; Lecture 4 shows it is $O(n^{\log_2 7})$. How one would discover these seven products is left open.

Proposition 3.3 [Strassen's identities].

$$XY = \begin{pmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{pmatrix}.$$

Proof. Expand each block, keeping the order of every matrix product, since matrix multiplication is not commutative. For the top-left block,

$$P_5 + P_4 - P_2 + P_6 = (AE + AH + DE + DH) + (DG - DE) - (AH + BH) \\ + (BG + BH - DG - DH) = AE + BG.$$

Similarly, $P_1 + P_2 = AF - AH + AH + BH = AF + BH$, $P_3 + P_4 = CE + DE + DG - DE = CE + DG$, and

$$P_1 + P_5 - P_3 - P_7 = (AF - AH) + (AE + AH + DE + DH) \\ - (CE + DE) - (AE + AF - CE - CF) = CF + DH.$$

□

3.4 The closest pair problem

Definition 3.4 [Closest pair]. Given a set $P = \{p_1, \ldots, p_n\}$ of $n \geq 2$ points in the plane, with Euclidean distance

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad \text{for } p_i = (x_i, y_i),\ p_j = (x_j, y_j),$$

find a pair of distinct points $p^*, q^* \in P$ minimizing $d(p, q)$.

For convenience, assume that all points have distinct x-coordinates and distinct y-coordinates. Brute-force search takes $\Theta(n^2)$ time. In one dimension, sorting in $O(n \log n)$ time and comparing adjacent points in $O(n)$ time suffices. The goal is an $O(n \log n)$ algorithm in two dimensions.

Sorting is still the first step: make copies P_x and P_y of the points sorted by x- and by y-coordinate, in $O(n \log n)$ time. This is not enough, because the closest pair need not be adjacent in either order. Divide and conquer provides the rest.

Algorithm 3.3: Closest pair

Input: Copies P_x, P_y of a point set P, $|P| = n \geq 2$, sorted by x- and by y-coordinate.
Output: A closest pair of points of P.

```
if n ≤ 3 then
    return a closest pair found by brute force
Q ← first ⌈n/2⌉ points of P_x (left half); R ← the remaining points
Form Q_x, R_x by splitting P_x, and Q_y, R_y by one scan of P_y
(p_1, q_1) ← ClosestPair(Q_x, Q_y)
(p_2, q_2) ← ClosestPair(R_x, R_y)
δ ← min{d(p_1, q_1), d(p_2, q_2)}
(p_3, q_3) ← ClosestSplitPair(P_x, P_y, δ)
return the best of (p_1, q_1), (p_2, q_2), and (p_3, q_3), if found
```

The scan of P_y sends each point to Q_y or R_y according to its x-coordinate, so all four lists take $O(n)$ time. A split pair has one point in Q and one in R. The key idea is that a split pair matters only

in the "unlucky case" where it is closer than the best pairs (p_1, q_1) and (p_2, q_2) returned by the recursive calls. Therefore ClosestSplitPair must only

1. run in $O(n)$ time, and
2. be correct whenever the closest pair of P is a split pair at distance less than δ.

Then $T(n) \le 2T(n/2) + O(n)$, as for merge sort, and the whole algorithm runs in $O(n \log n)$ time, including the initial sorting.

Algorithm 3.4: Closest split pair

Input: P_x, P_y, and $\delta > 0$ as above.
Output: The closest split pair at distance less than δ, if one exists.

$\bar{x} \leftarrow$ largest x-coordinate in the left half Q (the last point of Q_x)
$S_y \leftarrow$ points of P_y with x-coordinate in $[\bar{x} - \delta, \bar{x} + \delta]$, sorted by y-coordinate
best $\leftarrow \delta$; *bestpair* $\leftarrow$ none
for $i \leftarrow 1$ **to** $|S_y| - 1$ **do**
    for $j \leftarrow 1$ **to** $\min\{7, |S_y| - i\}$ **do**
        $p \leftarrow S_y[i]$; $q \leftarrow S_y[i + j]$
        if $d(p, q) <$ *best* **then**
            bestpair $\leftarrow (p, q)$; *best* $\leftarrow d(p, q)$
return *bestpair*

Finding $\bar{x}$ takes $O(1)$ time and filtering P_y keeps its y-order in $O(n)$ time. Each point is compared with at most the next seven points of S_y, so the loops take $O(n)$ time.

3.5 Correctness of the split-pair search

Theorem 3.5 [Correctness claim]. *Let $p \in Q$ and $q \in R$ be a split pair with $d(p, q) < \delta$. Then*

(A) p and q are members of S_y;

(B) p and q are at most 7 positions apart in S_y.

Proof of (A). Write $p = (x_1, y_1)$ and $q = (x_2, y_2)$. Since $d(p, q) < \delta$, both $|x_1 - x_2| < \delta$ and $|y_1 - y_2| < \delta$. Because $p \in Q$ and $q \in R$, we have $x_1 \le \bar{x} \le x_2$. Hence

$$\bar{x} - \delta \le x_2 - \delta < x_1 \le \bar{x} \qquad \text{and} \qquad \bar{x} \le x_2 < x_1 + \delta \le \bar{x} + \delta,$$

so both x-coordinates lie in $[\bar{x} - \delta, \bar{x} + \delta]$. □

For (B), draw eight $\delta/2 \times \delta/2$ boxes in two rows of four, centered horizontally at $\bar{x}$ and with bottom edge at height $\min\{y_1, y_2\}$. Assign shared box boundaries so that each point lies in one box, and each box lies on one side of the line $x = \bar{x}$.

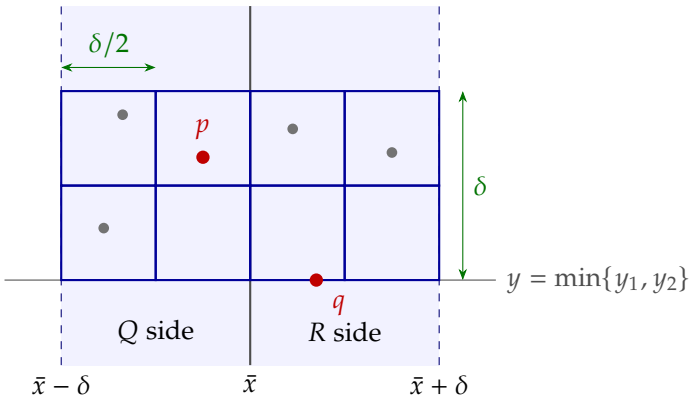


Figure 7: Schematic of the eight boxes in the strip that defines S_y. Every point of S_y with y-coordinate between those of p and q lies in a box, and no box contains two points.

Lemma 3.6. *Every point of S_y whose y-coordinate is between those of p and q, inclusive, lies in one of the eight boxes.*

Proof. The y-coordinates of p and q differ by less than δ, so such a point has height between $\min\{y_1, y_2\}$ and $\min\{y_1, y_2\} + \delta$. By the definition of S_y, its x-coordinate is between $\bar{x} - \delta$ and $\bar{x} + \delta$. These two ranges are exactly covered by the boxes. □

Lemma 3.7. *Each box contains at most one point of P.*

Proof. Suppose distinct points a, b lie in the same box. The box is on one side of $x = \bar{x}$, so a and b are both in Q or both in R. Its diagonal has length $\sqrt{2}\,\delta/2$, so

$$d(a, b) \leq \frac{\delta}{2}\sqrt{2} < \delta.$$

This contradicts the definition of δ as the smallest distance between a pair of points both in Q or both in R. □

Proof of (B). By the two lemmas, at most eight points of S_y, including p and q, have y-coordinates between those of p and q. Since S_y is sorted by y-coordinate, these are exactly the points from p to q in S_y. Hence the positions of p and q differ by at most 7. □

Corollary 3.8. *If the closest pair of P is a split pair,* ClosestSplitPair *finds it.*

Proof. If its distance is less than δ, theorem 3.5 places both points in S_y at most 7 positions apart, and the loops compare every such pair. If its distance equals δ, a recursive call already returned a pair at the minimum distance. □

Hence ClosestPair is correct: if the closest pair lies within Q or within R, a recursive call finds it;

otherwise it is a split pair, found by the split-pair search. Since every step outside the recursive calls takes $O(n)$ time, it runs in $O(n \log n)$ time.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Divide and Conquer Algorithms.

Lecture 4: The Master Method

Promising algorithmic ideas often need a mathematical analysis before we can tell whether they help. The master method is a "black box" that bounds the running times of many divide and conquer algorithms directly from their recurrences.

4.1 Recurrences for integer multiplication

The grade-school algorithm uses $\Theta(n^2)$ operations to multiply two n-digit numbers. For a recursive approach, write

$$x = 10^{n/2}a + b, \qquad y = 10^{n/2}c + d,$$

where a, b, c, d are $n/2$-digit numbers. Then

$$xy = 10^n ac + 10^{n/2}(ad + bc) + bd. \tag{4.1}$$

Let $T(n)$ be the maximum number of operations an algorithm needs to multiply two n-digit numbers. A recurrence expresses $T(n)$ in terms of the running times of the recursive calls.

- *Algorithm 1* recursively computes ac, ad, bc, bd, then evaluates equation (4.1) directly. With $T(1)$ at most a constant, for all $n > 1$,

$$T(n) \leq \underbrace{4T(n/2)}_{\text{recursive calls}} + \underbrace{O(n)}_{\text{work done here}}.$$

- *Algorithm 2* (Gauss's trick, the method of Lecture 1) recursively computes only ac, bd, and $(a + b)(c + d)$, and recovers $ad + bc = (a + b)(c + d) - ac - bd$. Its recurrence is

$$T(n) \leq 3T(n/2) + O(n).$$

4.2 The precise statement

Assume that all subproblems have equal size, and that the recurrence has the following form.

1. Base case: $T(n)$ is at most a constant for all sufficiently small n.
2. For all larger n,

$$T(n) \leq aT(n/b) + O(n^d),$$

where $a \geq 1$ is the number of recursive calls, $b > 1$ is the factor by which the input size shrinks,

and $d \geq 0$ is the exponent in the running time of the combine step. The parameters a, b, d do not depend on n.

Theorem 4.1 [Master method]. *For a recurrence of this form,*

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad \textit{(Case 1)}, \\ O(n^d) & \text{if } a < b^d \quad \textit{(Case 2)}, \\ O(n^{\log_b a}) & \text{if } a > b^d \quad \textit{(Case 3)}. \end{cases}$$

In Case 1 the base of the logarithm does not matter, since changing it only changes the leading constant. In Case 3 the base b appears in the exponent and does matter.

4.3 Examples

Recurrence	a	b	d	Case	Bound
Merge sort	2	2	1	1	$O(n \log n)$
Binary search of a sorted array	1	2	0	1	$O(\log n)$
Integer multiplication, Algorithm 1	4	2	1	3	$O(n^{\log_2 4}) = O(n^2)$
Integer multiplication, Gauss	3	2	1	3	$O(n^{\log_2 3}) = O(n^{1.59})$
Strassen's matrix multiplication	7	2	2	3	$O(n^{\log_2 7}) = O(n^{2.81})$
$T(n) \leq 2T(n/2) + O(n^2)$	2	2	2	2	$O(n^2)$

For merge sort, $b^d = 2 = a$. Binary search makes one recursive call on half the array and does constant extra work, so $b^d = 1 = a$. The first integer multiplication algorithm has $b^d = 2 < 4 = a$ and is no better than the grade-school algorithm. Gauss's trick has $b^d = 2 < 3 = a$ and beats it. Strassen's algorithm has $b^d = 4 < 7 = a$ and beats the naive iterative algorithm for matrix multiplication. In the last, fictitious recurrence, $b^d = 4 > 2 = a$. The exponents 1.59 and 2.81 are rounded values of $\log_2 3$ and $\log_2 7$.

4.4 The recursion tree

To prove the theorem, generalize the recursion-tree analysis of merge sort. Assume $T(1) \leq c$ and $T(n) \leq aT(n/b) + cn^d$ for one constant c, and that n is a power of b. The general case is similar but more tedious.

The tree has levels $j = 0, 1, \ldots, \log_b n$, where $\log_b n$ is the number of times n can be divided by b before reaching 1. Each node has a children, so level j has a^j subproblems, each of size n/b^j. The leaves are the base cases of size 1.

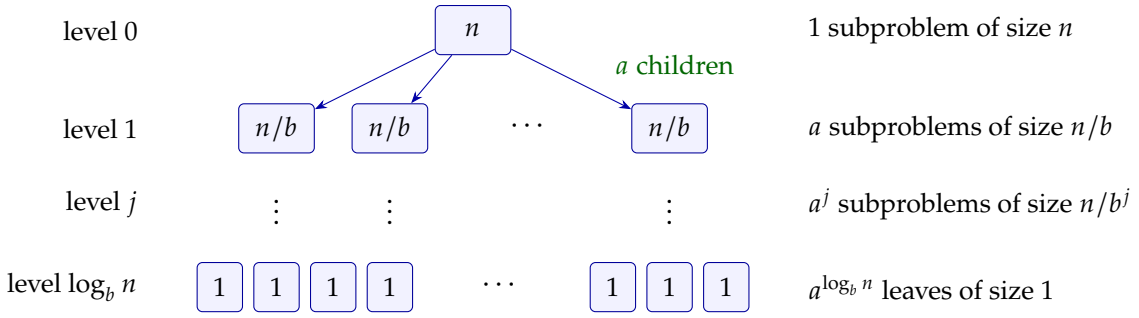


Figure 8: Schematic recursion tree for $T(n) \le aT(n/b) + cn^d$ with n a power of b. Only three of the a children of the root are drawn.

Ignoring work done inside recursive calls, the work at level j is at most

$$\underbrace{a^j}_{\substack{\text{number of} \\ \text{subproblems}}} \cdot c\left(\frac{n}{b^j}\right)^d = cn^d\left(\frac{a}{b^d}\right)^j.$$

At the leaves this is c per subproblem, matching the base case. Summing over all levels,

$$\text{total work} \le cn^d \sum_{j=0}^{\log_b n} \left(\frac{a}{b^d}\right)^j. \tag{4.2}$$

4.5 Intuition for the three cases

Read equation (4.2) as a contest between two forces: a is the RATE OF SUBPROBLEM PROLIFERATION (RSP), and b^d is the RATE OF WORK SHRINKAGE (RWS) per subproblem.

1. If RSP = RWS, every level does the same amount of work, as in merge sort. Expect $O(n^d \log n)$.
2. If RSP < RWS, the work decreases with the level, so most work is at the root. Expect $O(n^d)$.
3. If RSP > RWS, the work increases with the level, so most work is at the leaves. Expect $O(\#\text{leaves})$.

4.6 Completing the proof

Lemma 4.2 [BASIC SUMS FACT]. *For $r \neq 1$ and every integer $k \ge 0$,*

$$1 + r + r^2 + \cdots + r^k = \frac{r^{k+1} - 1}{r - 1}.$$

Proof. Induct on k. The case $k = 0$ is $1 = (r-1)/(r-1)$. If the identity holds for k, adding r^{k+1} gives $\left(r^{k+1} - 1 + r^{k+2} - r^{k+1}\right)/(r-1) = (r^{k+2} - 1)/(r-1)$. □

Two consequences hold for a constant r, uniformly in k:

- if $r < 1$, the sum is at most $1/(1-r)$, a constant, so the first term dominates;
- if $r > 1$, the sum is at most $r^{k+1}/(r-1) = r^k\left(1 + \frac{1}{r-1}\right)$, a constant times the last term, which dominates.

Proof of Theorem 4.1. Let $r = a/b^d$ in equation (4.2).
Case 1 ($a = b^d$). Every term of the sum equals 1, so the total work is at most $cn^d(\log_b n + 1) = O(n^d \log n)$.
Case 2 ($a < b^d$). Then $r < 1$, and by lemma 4.2 the sum is at most a constant independent of n. The total work is $O(n^d)$, dominated by the top level.
Case 3 ($a > b^d$). Then $r > 1$, and the sum is at most a constant times its largest term. So the total work is

$$O\left(n^d \left(\frac{a}{b^d}\right)^{\log_b n}\right) = O\left(a^{\log_b n}\right), \qquad \text{since } b^{-d \log_b n} = \left(b^{\log_b n}\right)^{-d} = n^{-d}.$$

Here $a^{\log_b n}$ is the number of leaves of the recursion tree. Finally, $a^{\log_b n} = n^{\log_b a}$, since both sides have logarithm base b equal to $(\log_b n)(\log_b a)$. The left form is more intuitive; the right form is simpler to apply. Thus $T(n) = O(n^{\log_b a})$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), The Master Method.

Lecture 5: QuickSort: Algorithm

QuickSort sorts by partitioning an array around one of its elements and recursing on the two sides. With random pivots it runs in $O(n \log n)$ time on average and works in place, using only a constant amount of extra memory beyond the input array.

The input is an array of n numbers, and the output is the same numbers in increasing order. Assume throughout that all entries are distinct.

5.1 Partitioning around a pivot

Definition 5.1 [Partition]. Given an array and one of its elements p, the pivot, a partition around p rearranges the array so that every entry left of p is less than p and every entry right of p is greater than p.

The two sides need not be sorted. For example, partitioning $3, 8, 2, 5, 1, 4, 7, 6$ around 3 may produce $2, 1, 3, 6, 7, 4, 5, 8$. Exactly the entries smaller than p precede it, so the pivot ends in its rightful position: the position it occupies in the sorted output.

Partitioning has two useful properties. It takes linear time with no extra memory, as shown below, and it reduces the problem size: the entries on each side can be sorted independently, and neither side contains the pivot.

With $O(n)$ extra memory, partitioning in $O(n)$ time is easy. Scan the input once, writing each entry less than p into a new array from the left end and each entry greater than p from the right end, then place p in the one remaining slot.

Algorithm 5.1: QuickSort, high-level description

Input: Array A of length n, with distinct entries.
Output: A rearranged into increasing order.

```
if n ≤ 1 then
    return
p ← ChoosePivot(A, n)
Partition A around p
Recursively sort the first part, the entries left of p
Recursively sort the second part, the entries right of p
```

The rule ChoosePivot is left open for now. Either part may be empty, so the base case includes $n = 0$.

5.2 In-place partitioning

Assume the pivot is the first entry of the subarray. Otherwise, first swap the pivot with the first entry. A single left-to-right scan then keeps the scanned entries partitioned: the pivot, then entries less than p, then entries greater than p, followed by the unscanned entries.

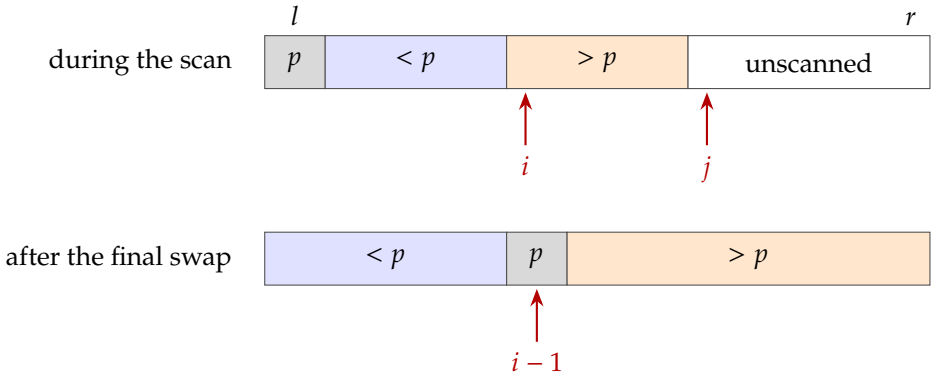


Figure 9: The subarray $A[l : r]$ during the scan and after the final swap. The index i marks the first entry greater than p, and j marks the next entry to examine.

Algorithm 5.2: Partition

Input: Subarray $A[l : r]$ with distinct entries; the pivot is $A[l]$.
Output: $A[l : r]$ partitioned around the pivot, and the pivot's final index.

```
1 p ← A[l]
2 i ← l + 1
3 for j ← l + 1 to r do
4 |   if A[j] < p then // if A[j] > p, do nothing
5 |   |   swap A[j] and A[i]
6 |   |   i ← i + 1
7 swap A[l] and A[i − 1]
8 return i − 1
```

The loop examines each of the $n - 1$ non-pivot entries once, with $O(1)$ work per entry, where $n = r - l + 1$. Thus Partition runs in $O(n)$ time. It only swaps entries, so it works in place.

Lemma 5.2 [Partition invariant]. *At the start of the iteration for each j, and when the loop ends with $j = r + 1$,*

$$A[l + 1], \ldots, A[i - 1] < p \qquad \text{and} \qquad A[i], \ldots, A[j - 1] > p.$$

Consequently Partition correctly partitions $A[l : r]$, and returns the final index of the pivot.

Proof. Use induction on j. Initially $i = j = l + 1$, so both ranges are empty. Suppose the invariant holds before examining $A[j]$. If $A[j] > p$, the second range simply grows to include position j. If $A[j] < p$, then $A[i]$ is either the first entry greater than p, which moves to position

j at the end of the second range, or $i = j$ and the swap does nothing. Either way, the entry less than p moves to position i, extending the first range, and incrementing i restores both conditions. When the loop ends, $A[l+1 : i-1]$ holds the entries less than p and $A[i : r]$ holds those greater than p. The final swap moves p to position $i-1$ and moves $A[i-1]$, which is either less than p or the pivot itself, to position l. Now everything left of position $i-1$ is less than p and everything right of it is greater. □

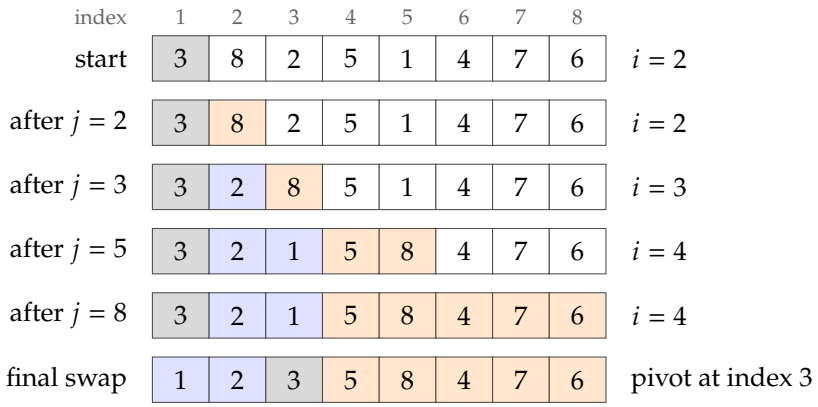


Figure 10: Partition on $3, 8, 2, 5, 1, 4, 7, 6$ with pivot 3 (gray). Blue entries are less than the pivot, orange entries are greater, and white entries are unscanned. Only $j = 3$ and $j = 5$ trigger swaps; the other iterations only extend the orange range.

In the in-place implementation, a call on $A[l : r]$ partitions it, obtains the pivot index q, and recurses on $A[l : q-1]$ and $A[q+1 : r]$. No combining step is needed.

5.3 Correctness

Let $P(n)$ be the assertion that QuickSort correctly sorts every input array of length n. Strong induction proves $P(n)$ for all $n \geq 1$: prove $P(1)$ directly, and for each $n \geq 2$ prove that $P(k)$ for all $k < n$ implies $P(n)$. The INDUCTIVE HYPOTHESIS is the assumption that $P(k)$ holds for every $k < n$.

Theorem 5.3 [CORRECTNESS OF QUICKSORT]. *For every $n \geq 1$, QuickSort correctly sorts every input array of length n, no matter how the pivots are chosen.*

Proof. An array of length one is already sorted, and QuickSort returns it unchanged, so $P(1)$ holds; an empty part is likewise returned unchanged. For the inductive step, fix $n \geq 2$ and an input array of length n. QuickSort first partitions around some pivot p. Let k_1 and k_2 be the lengths of the first and second parts. Since p belongs to neither part, $k_1, k_2 < n$. By the inductive hypothesis $P(k_1)$ and $P(k_2)$, the recursive calls sort both parts correctly. Every entry of the first part is less than p, which is less than every entry of the second part. Hence the whole array is sorted. □

5.4 The importance of the pivot

The running time of QuickSort depends on the quality of the pivot.

Example: Suppose ChoosePivot always selects the first entry, and the input is already sorted. Each pivot is the minimum, so the first part is empty and the second part has length $n-1$ and is still sorted. Partitioning a subarray of length m takes time proportional to m, so the running time is at least proportional to

$$n + (n-1) + (n-2) + \cdots + 1 \;\geq\; \frac{n}{2} \cdot \frac{n}{2} = \frac{n^2}{4},$$

because the first $n/2$ terms are each at least $n/2$. With the matching upper bound $n(n+1)/2$, the running time is $\Theta(n^2)$.

Example: Suppose instead that every recursive call magically chooses the median of its subarray as its pivot. Let $T(n)$ be the running time on arrays of length n. Both parts then have length about $n/2$, and choosing the pivot and partitioning take $\Theta(n)$ time, so

$$T(n) \leq 2T(n/2) + \Theta(n).$$

This is the merge-sort recurrence, so $T(n) = \Theta(n \log n)$.

5.5 Random pivots

The key question is how to choose pivots. The answer is to choose them randomly: in every recursive call, choose the pivot uniformly at random, with each entry of the subarray equally likely. The hope is that a random pivot is good enough sufficiently often.

Two observations support this. First, a 25–75 SPLIT, where each part has at most 3/4 of the entries, would already suffice. If every call achieved such a split, a subproblem at depth d of the recursion tree would have size at most $(3/4)^d n$, so the depth would be at most $\log_{4/3} n + 1 = O(\log n)$. The subarrays at any one depth are disjoint, so partitioning them all takes $O(n)$ total time per level, giving $O(n \log n)$ overall. Second, half of the entries give a 25–75 split or better: for n divisible by 4, any pivot whose rank lies between $n/4 + 1$ and $3n/4$.

Theorem 5.4 [QUICKSORT THEOREM]. *For every input array of length n, the average running time of QuickSort with random pivots is $O(n \log n)$.*

The bound holds for every input, with no assumptions on the data. The average is over the random choices made by the algorithm, namely its pivot choices, not over a distribution of inputs. Lecture 6 proves the theorem.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), QuickSort: Algorithm.

Lecture 6: QuickSort: Analysis

This lecture proves the QuickSort theorem: for every input array of length n, the average running time of QuickSort with uniformly random pivots is $O(n \log n)$. The average is over the pivot choices made by the algorithm; the input is arbitrary. The proof uses finite sample spaces, random variables, expectation, and linearity of expectation (Lecture 7).

6.1 Reducing running time to comparisons

Fix an input array A of length n with distinct entries. The sample space Ω is the set of all possible outcomes of the random choices in QuickSort, that is, all pivot sequences. For $\sigma \in \Omega$, let $C(\sigma)$ be the number of comparisons between two input entries made by QuickSort with pivot sequence σ, and let $RT(\sigma)$ be its running time.

Lemma 6.1 [Comparisons dominate]. *For $n \geq 2$, there is a constant c such that $RT(\sigma) \leq c\, C(\sigma)$ for every $\sigma \in \Omega$.*

Proof. A call on a subarray of length $m \geq 2$ chooses a pivot, swaps, and runs Partition, which makes $m - 1 \geq 1$ comparisons, each with $O(1)$ surrounding work. Hence, for some constant c', such a call does at most $c'm \leq 2c'(m-1)$ work beyond its recursive calls. A call on length 0 or 1 does $O(1)$ work and makes no comparisons, but it is one of at most two children of a call on length $m \geq 2$, so its cost can be charged to that parent's comparisons. Summing over all calls gives the bound. □

It therefore remains to prove $\mathbb{E}[C] = O(n \log n)$. The master method does not apply, because the subproblems are random and can be unbalanced.

6.2 Decomposing the comparison count

Let z_i denote the ith smallest entry of A; these labels refer to values, not positions. For indices $i < j$ and $\sigma \in \Omega$, let $X_{ij}(\sigma)$ be the number of times z_i and z_j are compared when QuickSort uses pivot sequence σ.

Any two entries are compared at most once. Every comparison involves the current pivot, and the pivot is excluded from both recursive calls, so it is never compared again. Thus each X_{ij} is an indicator random variable, taking only the values 0 and 1.

Every comparison is between some pair z_i, z_j with $i < j$, so

$$C(\sigma) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}(\sigma) \qquad \text{for every } \sigma \in \Omega.$$

By linearity of expectation, and since $\mathbb{E}[X_{ij}] = 0 \cdot \mathbb{P}[X_{ij} = 0] + 1 \cdot \mathbb{P}[X_{ij} = 1] = \mathbb{P}[X_{ij} = 1]$,

$$\mathbb{E}[C] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{P}[z_i, z_j \text{ get compared}]. \tag{6.1}$$

The random variable C is complicated, but each X_{ij} is simple.

This is an instance of a general DECOMPOSITION PRINCIPLE:

1. identify the random variable Y of interest;
2. express Y as a sum of indicator random variables, $Y = \sum_{e=1}^{m} X_e$;
3. apply linearity of expectation, $\mathbb{E}[Y] = \sum_{e=1}^{m} \mathbb{P}[X_e = 1]$, and analyze each of these probabilities.

6.3 The key insight

Proposition 6.2 [COMPARISON PROBABILITY]. *For all $i < j$,*

$$\mathbb{P}[z_i, z_j \textit{ get compared}] = \frac{2}{j - i + 1}.$$

Proof. Consider the $j - i + 1$ entries $z_i, z_{i+1}, \ldots, z_j$. As long as none of them has been chosen as a pivot, they all lie in the same recursive call: any pivot used so far is either smaller than all of them or larger than all of them, so partitioning sends all of them to the same side. Consider the first among $z_i, \ldots, z_j$ chosen as a pivot.

1. If it is z_i or z_j, then the other one is in the same subarray and is compared with the pivot during partitioning.
2. If it is some z_k with $i < k < j$, then $z_i < z_k < z_j$ are split into different recursive calls, so z_i and z_j are never compared.

When the first of them is chosen, the pivot is uniform over a subarray containing all $j - i + 1$ of them, so each is equally likely to be the first. Two of these $j - i + 1$ choices lead to case 1. □

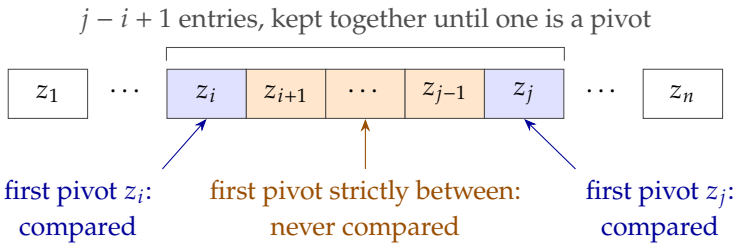


Figure 11: Entries in sorted order. Until one of $z_i, \ldots, z_j$ becomes a pivot, the whole group stays together. The comparison happens exactly when the first pivot from the group is one of its two endpoints.

Substituting into equation (6.1) gives

$$\mathbb{E}[C] = 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{1}{j-i+1}. \tag{6.2}$$

6.4 Final calculation

The double sum in equation (6.2) has $\Theta(n^2)$ terms, so it must be bounded more carefully than term by term. For each fixed i, the inner sum is

$$\sum_{j=i+1}^{n} \frac{1}{j-i+1} = \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-i+1} \leq \sum_{k=2}^{n} \frac{1}{k}.$$

There are fewer than n choices of i, so

$$\mathbb{E}[C] \leq 2n \sum_{k=2}^{n} \frac{1}{k}.$$

Lemma 6.3 [Harmonic bound]. $\displaystyle\sum_{k=2}^{n} \frac{1}{k} \leq \ln n.$

Proof. Since $1/x$ is decreasing, $1/k \leq 1/x$ for $x \in [k-1, k]$, so the term $1/k$ is the area of a width-one rectangle lying under the curve $1/x$ on $[k-1, k]$. Therefore

$$\sum_{k=2}^{n} \frac{1}{k} \leq \int_1^n \frac{dx}{x} = \ln x \Big|_1^n = \ln n - \ln 1 = \ln n. \qquad \square$$

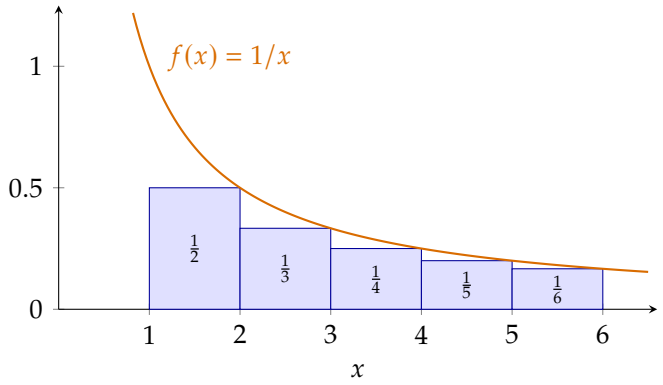


Figure 12: The rectangle of height $1/k$ on $[k-1, k]$ lies under $f(x) = 1/x$, shown here for $n = 6$. The total rectangle area $\sum_{k=2}^{n} 1/k$ is at most the area under the curve from 1 to n.

Theorem 6.4 [QuickSort theorem]. *For every input array of length n, the expected number of comparisons of QuickSort with random pivots satisfies $\mathbb{E}[C] \leq 2n \ln n$. Hence its average running time is $O(n \log n)$.*

Proof. Combine the bound $\mathbb{E}[C] \leq 2n \sum_{k=2}^{n} 1/k$ with the harmonic bound. By the comparison lemma, $\mathbb{E}[RT] \leq c\,\mathbb{E}[C] \leq 2cn \ln n$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), QuickSort: Analysis.

Lecture 7: Probability Review

The analysis of randomized algorithms such as QuickSort uses a small amount of discrete probability. Two running examples recur: rolling two dice, and choosing a uniformly random pivot in the outermost call of QuickSort on an array of length n with distinct entries.

7.1 Sample spaces and events

Definition 7.1 [Sample space]. A sample space Ω is the set of all possible outcomes. In algorithms Ω is usually finite. Each outcome $i \in \Omega$ has a probability $p(i) \geq 0$, and

$$\sum_{i \in \Omega} p(i) = 1.$$

For two dice, $\Omega = \{(1,1), (2,1), \ldots, (6,6)\}$ consists of the 36 ordered pairs, each with probability $1/36$. For a random pivot, $\Omega = \{1, 2, \ldots, n\}$ is the index of the pivot, and $p(i) = 1/n$ for every $i \in \Omega$.

Definition 7.2 [Event]. An event is a subset $S \subseteq \Omega$. Its probability is

$$\mathbb{P}[S] = \sum_{i \in S} p(i).$$

Example: The event that the two dice sum to 7 is $S = \{(1,6), (2,5), (3,4), (4,3), (5,2), (6,1)\}$, so $\mathbb{P}[S] = 6/36 = 1/6$.

Example: Let n be divisible by 4. The event that the pivot gives a 25–75 split or better contains the pivots from the $(n/4 + 1)$th smallest entry to the $(3n/4)$th smallest entry. It has $n/2$ outcomes, so $\mathbb{P}[S] = (n/2)/n = 1/2$.

7.2 Random variables and expectation

Definition 7.3 [Random variable]. A random variable is a real-valued function $X : \Omega \to \mathbb{R}$.

Examples are the sum of the two dice, and the size of the subarray passed to the first recursive call of QuickSort.

Definition 7.4 [Expectation]. The expectation of a random variable X is its average value, weighted by probability:

$$\mathbb{E}[X] = \sum_{i \in \Omega} X(i)\, p(i).$$

Example: Let X be the size of the subarray passed to the first recursive call. If the pivot is the ith smallest entry, which happens with probability $1/n$, the first part has $i-1$ entries. Hence

$$\mathbb{E}[X] = \frac{1}{n}\cdot 0 + \frac{1}{n}\cdot 1 + \cdots + \frac{1}{n}\cdot(n-1) = \frac{1}{n}\cdot\frac{(n-1)n}{2} = \frac{n-1}{2}.$$

7.3 Linearity of expectation

Theorem 7.5 [Linearity of expectation]. *Let $X_1, \ldots, X_n$ be random variables defined on the same sample space Ω. Then*

$$\mathbb{E}\Big[\sum_{j=1}^{n} X_j\Big] = \sum_{j=1}^{n} \mathbb{E}[X_j].$$

Proof. Expand each expectation and reverse the order of the two finite sums:

$$\sum_{j=1}^{n}\mathbb{E}[X_j] = \sum_{j=1}^{n}\sum_{i\in\Omega} X_j(i)p(i) = \sum_{i\in\Omega} p(i)\sum_{j=1}^{n} X_j(i) = \mathbb{E}\Big[\sum_{j=1}^{n} X_j\Big]. \qquad \square$$

Crucially, the identity holds even when the X_j are not independent. It would fail in general if sums were replaced by products.

Example: Let X_1, X_2 be the values of the two dice. Each has $\mathbb{E}[X_j] = \frac{1}{6}(1+2+\cdots+6) = 3.5$, so the expected sum is $\mathbb{E}[X_1 + X_2] = 3.5 + 3.5 = 7$.

Example: *Load balancing.* Assign each of n processes to one of n servers, chosen uniformly at random. The sample space consists of all n^n assignments, each equally likely. Let Y be the number of processes assigned to the first server, and let $X_j = 1$ if process j is assigned to the first server and $X_j = 0$ otherwise. Then $Y = \sum_{j=1}^{n} X_j$, and each X_j is an indicator random variable with $\mathbb{P}[X_j = 1] = 1/n$. Thus

$$\mathbb{E}[Y] = \sum_{j=1}^{n}\mathbb{E}[X_j] = \sum_{j=1}^{n}\big(0\cdot\mathbb{P}[X_j=0] + 1\cdot\mathbb{P}[X_j=1]\big) = \sum_{j=1}^{n}\frac{1}{n} = 1.$$

7.4 Conditional probability

Definition 7.6 [Conditional probability]. For events $X, Y \subseteq \Omega$ with $\mathbb{P}[Y] > 0$, the conditional probability of X given Y is

$$\mathbb{P}[X \mid Y] = \frac{\mathbb{P}[X \cap Y]}{\mathbb{P}[Y]}.$$

Conditioning on Y restricts attention to the outcomes in Y, rescaled to total probability one.

Example: Roll two fair dice. Let X be the event that at least one die shows 1, and Y the event that the sum is 7. Then $X \cap Y = \{(1,6),(6,1)\}$, so

$$\mathbb{P}[X \mid Y] = \frac{2/36}{6/36} = \frac{1}{3}.$$

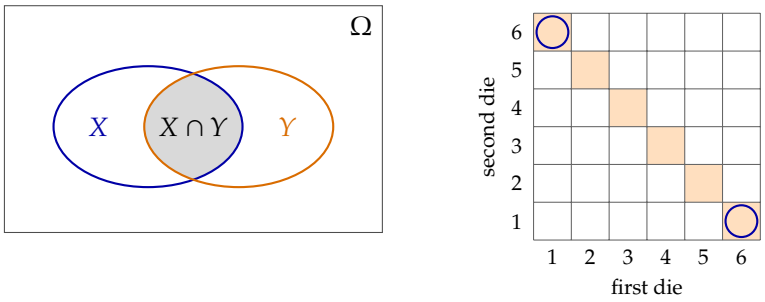


Figure 13: Left: events X and Y in a sample space Ω. Right: the 36 outcomes for two dice. The six outcomes with sum 7 form Y; the two circled outcomes form $X \cap Y$.

7.5 Independence

Definition 7.7 [Independent events]. Events $X, Y \subseteq \Omega$ are INDEPENDENT if and only if

$$\mathbb{P}[X \cap Y] = \mathbb{P}[X] \cdot \mathbb{P}[Y].$$

When $\mathbb{P}[Y] > 0$, dividing by $\mathbb{P}[Y]$ shows that this holds if and only if $\mathbb{P}[X \mid Y] = \mathbb{P}[X]$. Similarly, when $\mathbb{P}[X] > 0$, it holds if and only if $\mathbb{P}[Y \mid X] = \mathbb{P}[Y]$. Independence can be a very subtle concept, and intuition about it is often incorrect.

Definition 7.8 [Independent random variables]. Random variables A, B defined on Ω are independent if and only if the events $\{A = a\}$ and $\{B = b\}$ are independent for all values a, b, that is,

$$\mathbb{P}[A = a \text{ and } B = b] = \mathbb{P}[A = a] \cdot \mathbb{P}[B = b].$$

Proposition 7.9 [Expectation of a product]. *If A and B are independent, then $\mathbb{E}[AB] = \mathbb{E}[A] \cdot \mathbb{E}[B]$.*

Proof. Group outcomes by the values a of A and b of B, then use independence to factor each probability:

$$\begin{aligned}\mathbb{E}[AB] &= \sum_{a,b} ab\, \mathbb{P}[A = a \text{ and } B = b] = \sum_{a,b} ab\, \mathbb{P}[A = a]\, \mathbb{P}[B = b] \\ &= \Big(\sum_{a} a\, \mathbb{P}[A = a]\Big)\Big(\sum_{b} b\, \mathbb{P}[B = b]\Big) = \mathbb{E}[A] \cdot \mathbb{E}[B].\end{aligned}$$

□

Example: Let $X_1, X_2 \in \{0, 1\}$ be random bits and $X_3 = X_1 \oplus X_2$ their exclusive or. Formally, writing outcomes as $X_1X_2X_3$,

$$\Omega = \{000,\ 101,\ 011,\ 110\},$$

each equally likely. The pair (X_1, X_3) takes each of the four values in $\{0, 1\}^2$ on exactly one outcome, so each joint probability is $1/4 = \frac{1}{2} \cdot \frac{1}{2}$, and X_1, X_3 are independent.

However, X_1X_3 and X_2 are not independent. If they were, the product rule would give $\mathbb{E}[X_1X_2X_3] = \mathbb{E}[X_1X_3] \cdot \mathbb{E}[X_2]$. But $X_1X_2X_3 = 0$ on every outcome, so the left side is 0, while

$$\mathbb{E}[X_1X_3] \cdot \mathbb{E}[X_2] = \mathbb{E}[X_1]\,\mathbb{E}[X_3] \cdot \mathbb{E}[X_2] = \frac{1}{4} \cdot \frac{1}{2} = \frac{1}{8},$$

using the independence of X_1 and X_3.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Probability Review.

Lecture 8: Linear-Time Selection

Selection asks for one element of a given rank, rather than the whole sorted order. Sorting solves it in $O(n \log n)$ time; adapting QuickSort's partitioning solves it in linear time, first in expectation with random pivots and then deterministically.

8.1 The selection problem

Definition 8.1 [Selection]. Given an array A of n distinct numbers and an integer i with $1 \le i \le n$, return the ith order statistic: the ith smallest element of A.

Distinctness is assumed for simplicity. The median is the case $i = (n+1)/2$ for odd n and $i = n/2$ for even n. For example, the array $(10, 8, 2, 4)$ has third order statistic 8.

Merge sort followed by reading position i of the sorted array solves the problem in $O(n \log n)$ time. The last subsection shows that no comparison-based sorting algorithm is asymptotically faster, so a linear algorithm must avoid sorting.

8.2 Partitioning and randomized selection

To partition an array around a chosen pivot element p, rearrange it so that every element left of p is smaller than p and every element right of p is larger. The two sides need not be sorted. Partitioning takes linear time and puts p in its "rightful position": if it lands at position j, then p is the jth order statistic.

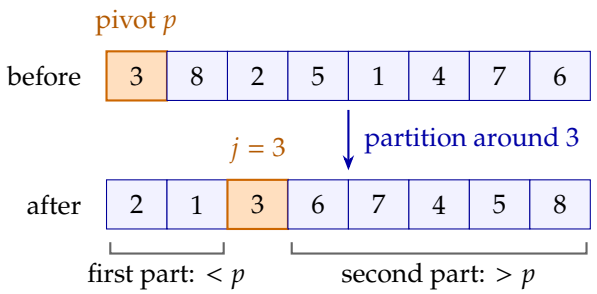


Figure 14: Partitioning $(3, 8, 2, 5, 1, 4, 7, 6)$ around the pivot 3. The pivot ends at position $j = 3$, its rank; the elements before it are smaller and the elements after it are larger, in no particular order.

Only one side of the pivot can contain the ith order statistic. If $j > i$, it is the ith smallest element of the first part. If $j < i$, then the first j positions hold the j smallest elements, so the target is the

$(i-j)$th smallest element of the second part. For instance, when seeking the 5th order statistic of 10 elements and the pivot lands in position 3, recurse on the right side for its 2nd order statistic.

Algorithm 8.1: Randomized selection, RSelect(A, n, i)

Input: Array $A[1:n]$ of distinct numbers and an integer $1 \le i \le n$.
Output: The ith order statistic of A.

```
if n = 1 then
    return A[1]
Choose a pivot p from A uniformly at random
Partition A around p; let j be the new position of p
if j = i then
    return p
if j > i then
    return RSelect(A[1 : j − 1], j − 1, i)
else
    return RSelect(A[j + 1 : n], n − j, i − j)
```

Proposition 8.2 [Correctness of RSelect]. *RSelect always returns the ith order statistic.*

Proof. Induct on n. The case $n = 1$ is immediate. For $n \ge 2$, the pivot is the jth order statistic. If $j = i$ it is the answer. Otherwise the recursive call receives a shorter array that, by the observation above, contains the answer at the stated rank; the induction hypothesis finishes the proof. □

The running time depends on the quality of the pivots. With the worst possible pivots it is $\Theta(n^2)$: if $i = n/2$ and the pivot is always the minimum, each of the first $n/2$ calls does $\Omega(n)$ partitioning work on an array that shrinks by only one element.

The ideal pivot is the median, which halves the array. That would give $T(n) \le T(n/2) + O(n)$, so $T(n) = O(n)$ by the master method ($a = 1 < b^d = 2$). Computing the median is the original problem, so this is circular. The hope is that a random pivot is good enough often enough.

Theorem 8.3 [RSelect]. *For every input array of length n, the expected running time of RSelect is $O(n)$.*

No assumption is made on the data. The expectation is over the random pivot choices made by the algorithm.

8.3 Analysis of randomized selection

Outside its recursive call, RSelect performs at most cn operations on an array of length n, for a constant $c > 0$, mostly from partitioning.

Track progress in phases. RSelect is in phase j when the current array has length between $(3/4)^{j+1}n$ and $(3/4)^j n$. Let X_j be the number of recursive calls made during phase j. Each such call works on

an array of length at most $(3/4)^j n$, so

$$\text{running time of RSelect} \le \sum_{\text{phases } j} X_j \cdot c \left(\tfrac{3}{4}\right)^j n. \tag{8.1}$$

Lemma 8.4 [Phases are short]. *For every phase j, $\mathbb{E}[X_j] \le 2$.*

Proof. If a call chooses a pivot giving a 25–75 split or better, the next array has at most 75% of the current length, so the current phase ends. Such a split occurs when the pivot's rank lies in the middle half of the current array, which has probability $1/2$ (ignoring rounding) independently of earlier choices. Call a good pivot "heads" and a bad one "tails". Then X_j is at most the number N of fair coin flips needed to see the first heads. This N is a geometric random variable. The first flip is always made; with probability $1/2$ it is tails and the process starts afresh. Hence

$$\mathbb{E}[N] = 1 + \tfrac{1}{2}\mathbb{E}[N], \qquad \text{so} \qquad \mathbb{E}[N] = 2.$$

Therefore $\mathbb{E}[X_j] \le \mathbb{E}[N] = 2$. □

Proof of the RSelect theorem. Take expectations in (8.1), use linearity of expectation, and bound the geometric series:

$$\mathbb{E}[\text{running time}] \le cn \sum_j \left(\tfrac{3}{4}\right)^j \mathbb{E}[X_j] \le 2cn \sum_{j \ge 0} \left(\tfrac{3}{4}\right)^j \le 2cn \cdot \frac{1}{1 - 3/4} = 8cn = O(n).$$ □

8.4 Deterministic selection

To guarantee a good pivot without randomness, use the median of medians. Break A into $n/5$ groups of 5 elements, sort each group, copy the $n/5$ group medians (middle elements) into a new array C, and recursively compute the median of C. The rest of the algorithm is unchanged. DSelect therefore makes two recursive calls: one to choose the pivot and one on a side of the partition.

Sorting the groups takes linear time. By the merge-sort bound $6m(\log_2 m + 1)$ with $m = 5$ and $\log_2 5 \le 3$, one group costs at most $6 \cdot 5 \cdot 4 = 120$ operations, so all $n/5$ groups cost at most $24n = O(n)$.

Algorithm 8.2: Deterministic selection, DSelect(A, n, i)

Input: Array $A[1:n]$ of distinct numbers and an integer $1 \le i \le n$.
Output: The ith order statistic of A.

```
if n = 1 then
    return A[1]
Break A into groups of 5 and sort each group
C ← the n/5 middle elements of the groups
p ← DSelect(C, n/5, n/10)                          // median of C
Partition A around p; let j be the new position of p
if j = i then
    return p
if j > i then
    return DSelect(A[1 : j − 1], j − 1, i)
else
    return DSelect(A[j + 1 : n], n − j, i − j)
```

Theorem 8.5 [DSelect]. *For every input array of length n, DSelect runs in $O(n)$ time.*

In practice DSelect is not as good as RSelect: its constants are worse and it does not work in place. The algorithm is due to Blum, Floyd, Pratt, Rivest, and Tarjan (1973).

8.5 Analysis of deterministic selection

Let $T(n)$ be the maximum running time of DSelect on an array of length n. There is a constant $c \ge 1$ such that $T(1) = 1$ and

$$T(n) \le cn + T(n/5) + T(?).$$

Here cn covers sorting the groups and partitioning, $T(n/5)$ is the call that chooses the pivot, and $T(?)$ is the call on one side of the partition, whose size depends on how good the pivot is.

Lemma 8.6 [Key lemma]. *The second recursive call of DSelect is on an array of length at most roughly $7n/10$.*

Proof. Let $k = n/5$ be the number of groups and x_i the ith smallest group median, so the pivot is $x_{k/2}$. The goal is to show that at least 30% of the input is smaller than $x_{k/2}$ and at least 30% is larger. Imagine the elements laid out in a grid: each column is a group, sorted increasing from bottom to top, and the columns are ordered left to right by their medians $x_1 < x_2 < \cdots < x_k$. The pivot $x_{k/2}$ is larger than the medians $x_1, \ldots, x_{k/2-1}$, and each median is larger than the two elements below it in its column. So $x_{k/2}$ exceeds 3 of the 5 elements (60%) in roughly half of the groups, hence about $\frac{3}{5} \cdot \frac{1}{2} = 30\%$ of A. Symmetrically, $x_{k/2}$ is smaller than the medians $x_{k/2+1}, \ldots, x_k$ and the two elements above each of them, again about 30% of A. Whichever side the second call recurses on, it excludes at least 30% of the input. □

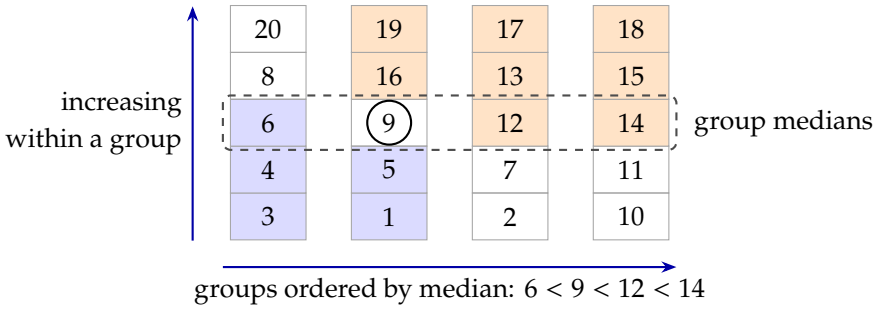


Figure 15: The grid for the input $(7, 2, 17, 12, 13,\ 8, 20, 4, 6, 3,\ 19, 1, 9, 5, 16,\ 10, 15, 18, 14, 11)$. The group medians are $12, 6, 9, 14$, and the pivot is their median 9 (circled). Blue cells are known to be smaller than the pivot; orange cells are known to be larger.

The recurrence becomes $T(1) = 1$ and

$$T(n) \le cn + T(n/5) + T(7n/10).$$

The subproblems have different sizes, so the master method does not apply. Instead, guess that $T(n) \le an$ for a constant a independent of n, and check it.

Proposition 8.7 [Solving the recurrence]. *With $a = 10c$, $T(n) \le an$ for all $n \ge 1$. Hence DSelect runs in $O(n)$ time.*

Proof. Induct on n. Base case: $T(1) = 1 \le a$, since $a \ge 1$. For $n > 1$, assume $T(k) \le ak$ for all $k < n$. Then

$$T(n) \le cn + T(n/5) + T(7n/10) \le cn + a\frac{n}{5} + a\frac{7n}{10} = n\left(c + \frac{9a}{10}\right) = an,$$

where the last step uses $c = a/10$. The argument works because the two subproblem sizes total $\frac{1}{5} + \frac{7}{10} = \frac{9}{10}$ of the input, strictly less than all of it. □

8.6 A sorting lower bound

A comparison-based sorting algorithm accesses the input elements only through comparisons; it is a general-purpose method. Merge sort, QuickSort, and heap sort are comparison-based. Bucket sort (suited to data drawn from known distributions), counting sort (small integers), and radix sort (medium-size integers) are not.

Theorem 8.8 [Sorting lower bound]. *Every comparison-based sorting algorithm has worst-case running time $\Omega(n \log n)$.*

The proof below assumes a deterministic algorithm; the lower bound extends to randomized

algorithms.

Proof. Fix a comparison-based algorithm and a length n. Consider the $n!$ input arrays that contain $\{1, 2, \ldots, n\}$ in some order. Suppose the algorithm always correctly sorts these inputs using at most k comparisons. The execution of a comparison-based algorithm is determined by the outcomes of its comparisons. With at most k binary outcomes, there are at most 2^k distinct executions across all $n!$ inputs. If $2^k < n!$, then by the pigeonhole principle two distinct inputs produce identical executions. Both are then rearranged in exactly the same way, so at most one of them comes out sorted, a contradiction. Hence

$$2^k \geq n! \geq \left(\frac{n}{2}\right)^{n/2},$$

where the second inequality keeps only the largest $n/2$ factors of $n!$, each at least $n/2$. Taking logarithms,

$$k \geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n).$$

□

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Linear-Time Selection.

Lecture 9: Graphs and the Contraction Algorithm

Graphs give a new domain for randomized algorithms. After the basic vocabulary and two representations, a simple random contraction algorithm finds a minimum cut with probability at least $1/n^2$; repetition makes failure unlikely, and the same analysis bounds how many minimum cuts a graph can have.

9.1 Graphs and cuts

A graph $G = (V, E)$ has two ingredients: vertices (or nodes) V, and edges E, which are pairs of vertices. An edge is undirected if the pair is unordered and directed (an arc) if it is ordered, from its tail to its head. Graphs model road networks, the Web, social networks, precedence constraints, and more.

Definition 9.1 [Cut]. A cut of a graph (V, E) is a partition of V into two nonempty sets A and B. The crossing edges of a cut (A, B) are the edges with one endpoint in each of A and B (undirected case), or with tail in A and head in B (directed case).

Each vertex can go to either side, and both sides must be nonempty, so a graph with n vertices has $2^n - 2$ ordered cuts (A, B): roughly 2^n.

Definition 9.2 [Minimum cut problem]. Given an undirected graph $G = (V, E)$, possibly with parallel edges, compute a cut with the fewest crossing edges, a minimum cut.

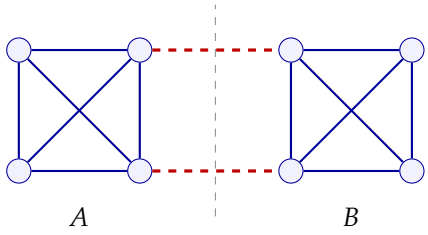


Figure 16: Two complete graphs on four vertices joined by two edges. The cut separating them has 2 crossing edges (dashed), which is the minimum.

Minimum cuts identify network bottlenecks and weaknesses, and help detect communities in social networks. In image segmentation, the input is a graph of pixels with edge weights, where a large weight on (u, v) means u and v are expected to come from the same object; repeated minimum cuts can identify the primary objects in the picture.

9.2 Representing graphs

Throughout, n is the number of vertices and m the number of edges. A connected undirected graph with no parallel edges has between $n-1$ and $n(n-1)/2$ edges. In most (but not all) applications, m is $\Omega(n)$ and $O(n^2)$. In a SPARSE graph m is $O(n)$ or close to it; in a DENSE graph m is closer to $\Theta(n^2)$.

The ADJACENCY MATRIX represents G by an $n \times n$ 0-1 matrix with $A_{ij} = 1$ exactly when G has an i-j edge. Variants store the number of i-j edges (for parallel edges), the weight of the i-j edge, or, for directed graphs, $A_{ij} = +1$ for an arc from i to j and -1 for an arc from j to i. It needs $\Theta(n^2)$ space regardless of m.

ADJACENCY LISTS store the following ingredients:

Ingredient	Space
array (or list) of vertices	$\Theta(n)$
array (or list) of edges	$\Theta(m)$
each edge points to its endpoints	$\Theta(m)$
each vertex points to its incident edges	$\Theta(m)$
total	$\Theta(m+n)$

The last two rows are in one-to-one correspondence: each edge-endpoint incidence is recorded once from each side. The total $\Theta(m+n)$ is the same as $\Theta(\max\{m,n\})$. Which representation is better depends on the graph's density and the operations needed; this course focuses on adjacency lists.

9.3 The random contraction algorithm

CONTRACTING an edge (u,v) merges u and v into a single vertex, which keeps all edges of both; edges between u and v become self-loops. Parallel edges can appear, which is why the problem allows them.

Algorithm 9.1: Random contraction (Karger, early 1990s)

Input: Undirected graph $G = (V, E)$, possibly with parallel edges.
Output: A cut of G.

```
while more than 2 vertices remain do
    Pick a remaining edge (u, v) uniformly at random
    Contract u and v into a single vertex
    Remove self-loops
return the cut whose sides are the vertex sets merged into the final two vertices
```

Each remaining vertex stands for the set of original vertices merged into it. The output depends on the random choices, and it need not be a minimum cut.

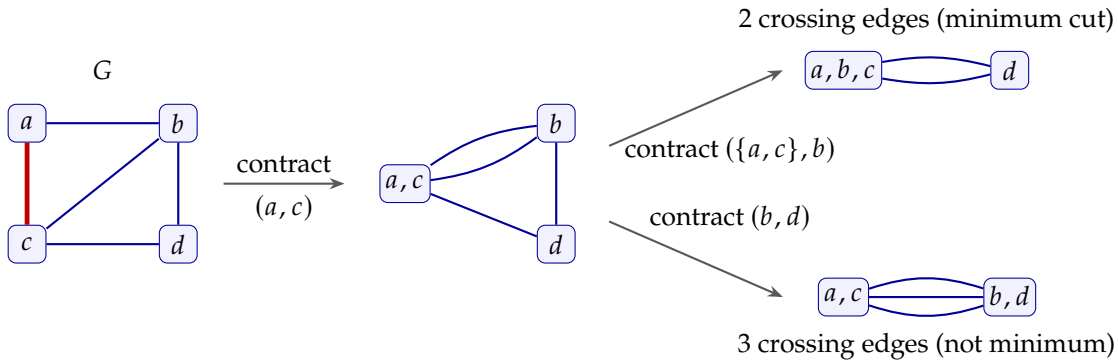


Figure 17: Two runs on the same graph, both contracting (a, c) first (thick edge). Contracting one of the two edges between $\{a, c\}$ and b next yields the cut $(\{a, b, c\}, \{d\})$ with 2 crossing edges, a minimum cut. Contracting (b, d) instead yields $(\{a, c\}, \{b, d\})$ with 3 crossing edges, which is not a minimum cut.

9.4 Success probability of one run

Fix a graph $G = (V, E)$ with n vertices and m edges, and fix a minimum cut (A, B). Let k be its number of crossing edges and F the set of these edges.

If an edge of F is ever contracted, an endpoint in A and an endpoint in B are merged, so the output cannot be (A, B). If only edges inside A or inside B are contracted, every merged vertex lies entirely in A or entirely in B, and the final two vertices must therefore be A and B. Thus

$$\mathbb{P}[\text{output is } (A, B)] = \mathbb{P}[\text{no edge of } F \text{ is ever contracted}].$$

Each iteration reduces the number of vertices by one, so there are $n - 2$ iterations. Let S_i be the event that an edge of F is contracted in iteration i. The goal is to bound $\mathbb{P}[\neg S_1 \wedge \neg S_2 \wedge \cdots \wedge \neg S_{n-2}]$.

Lemma 9.3 [First iteration]. $\mathbb{P}[S_1] = k/m \leq 2/n$.

Proof. The first edge is uniform among m edges, k of which cross (A, B), so $\mathbb{P}[S_1] = k/m$. Every vertex v defines the cut $(\{v\}, V - \{v\})$, whose crossing edges are the $\deg(v)$ edges incident to v. Since (A, B) is a minimum cut, $\deg(v) \geq k$ for every v. Summing degrees counts each edge twice, so $2m = \sum_v \deg(v) \geq kn$, that is, $m \geq kn/2$. Hence $k/m \leq 2/n$. □

The same argument applies to the contracted graphs. Each vertex of a contracted graph corresponds to a set of original vertices, which defines a cut of G with at least k crossing edges, so every degree in the contracted graph is at least k. Given that no edge of F has yet been contracted, all k edges of F remain.

Theorem 9.4 [Success probability]. *The contraction algorithm outputs the fixed minimum cut* (A, B) *with probability at least* $2/(n(n-1)) \geq 1/n^2$.

Proof. Before iteration i, the contracted graph has $n - i + 1$ vertices, each of degree at least k, so at least $k(n - i + 1)/2$ edges remain. Given $\neg S_1 \wedge \cdots \wedge \neg S_{i-1}$, all k edges of F remain, so

$$\mathbb{P}[\neg S_i \mid \neg S_1 \wedge \cdots \wedge \neg S_{i-1}] = 1 - \frac{k}{\#\text{ remaining edges}} \geq 1 - \frac{2}{n-i+1}.$$

For $i = 2$ this gives $1 - 2/(n-1)$. Multiplying the conditional probabilities,

$$\begin{aligned}\mathbb{P}[\neg S_1 \wedge \cdots \wedge \neg S_{n-2}] &= \mathbb{P}[\neg S_1]\,\mathbb{P}[\neg S_2 \mid \neg S_1] \cdots \mathbb{P}[\neg S_{n-2} \mid \neg S_1 \wedge \cdots \wedge \neg S_{n-3}] \\ &\geq \left(1 - \frac{2}{n}\right)\left(1 - \frac{2}{n-1}\right) \cdots \left(1 - \frac{2}{4}\right)\left(1 - \frac{2}{3}\right) \\ &= \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \frac{2}{4} \cdot \frac{1}{3} = \frac{2}{n(n-1)} \geq \frac{1}{n^2}.\end{aligned}$$

The product telescopes: each numerator cancels the denominator two factors later, leaving $2 \cdot 1$ over $n(n-1)$. □

The success probability is low, but nontrivial, since a graph has roughly 2^n cuts.

9.5 Repeated trials

Run the basic algorithm N times independently and remember the smallest cut found. Let T_i be the event that the cut (A, B) is found on the ith try. The T_i are independent by construction, so

$$\mathbb{P}[\text{all } N \text{ trials fail}] = \mathbb{P}[\neg T_1 \wedge \cdots \wedge \neg T_N] = \prod_{i=1}^{N} \mathbb{P}[\neg T_i] \leq \left(1 - \frac{1}{n^2}\right)^N.$$

The calculus fact $1 + x \leq e^x$ for every real x bounds this quantity.

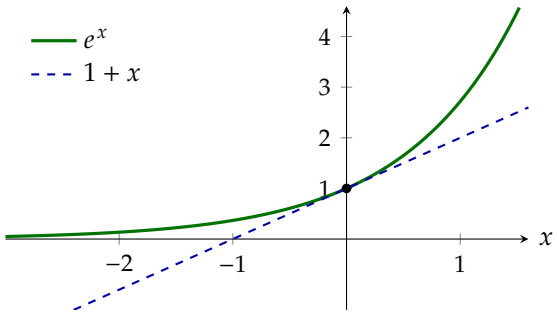


Figure 18: The line $1 + x$ lies below e^x, touching it only at $x = 0$.

With $x = -1/n^2$,

$$\mathbb{P}[\text{all fail}] \leq \left(e^{-1/n^2}\right)^N.$$

Taking $N = n^2$ trials gives failure probability at most $1/e$; taking $N = n^2 \ln n$ gives at most

$(1/e)^{\ln n} = 1/n$. The resulting running time is polynomial in n and m but slow, $\Omega(n^2 m)$. More ideas give large speedups, to roughly $O(n^2)$.

9.6 Counting minimum cuts

A graph can have several minimum cuts. For example, a tree with n vertices has $n - 1$ minimum cuts: removing any one edge splits the tree into two pieces, giving a cut with one crossing edge.

Theorem 9.5 [Number of minimum cuts]. *The largest number of minimum cuts that a graph with n vertices can have is $\binom{n}{2} = n(n-1)/2$.*

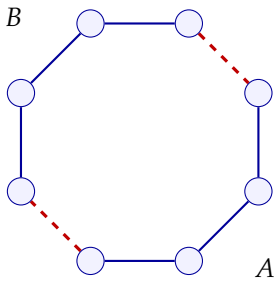


Figure 19: The 8-cycle. Removing any two edges, such as the dashed ones, splits it into two paths A and B, giving a minimum cut with 2 crossing edges.

Proof. Lower bound. Consider the n-cycle. Every cut crosses the cycle at least twice, since traversing the cycle from a vertex in A must leave A and return. Each pair of the n edges defines a cut with exactly these two crossing edges: removing them splits the cycle into two paths, which form the two sides. Different pairs give different cuts, so the n-cycle has at least $\binom{n}{2}$ minimum cuts.

Upper bound. Let $(A_1, B_1), \ldots, (A_t, B_t)$ be the minimum cuts of a graph with n vertices. The analysis of a single run of the contraction algorithm applies to each fixed minimum cut, so for every $i = 1, \ldots, t$,

$$\mathbb{P}[\text{output} = (A_i, B_i)] \geq \frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}}.$$

These events are disjoint, since one run outputs only one cut. Their probabilities therefore sum to at most 1:

$$\frac{t}{\binom{n}{2}} \leq 1, \qquad \text{so} \qquad t \leq \binom{n}{2}. \qquad \square$$

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Graphs and the Contraction Algorithm.

Lecture 10: Graph Search and Connectivity

Graph search answers questions such as whether a network is connected, so that every vertex can reach every other. In a graph of actors joined when they share a film, the path Jon Hamm, Colin Firth, Kevin Bacon gives Jon Hamm a Bacon number of 2. Search also underlies driving directions, planning (for a Sudoku puzzle, nodes are partially completed puzzles and arcs fill in one new square), and computing the pieces of a graph, used in clustering and in studying the structure of the Web. Throughout, $G = (V, E)$ has $n = |V|$ vertices and $m = |E|$ edges, stored as adjacency lists.

10.1 Generic graph search

The goals are to find everything findable from a given start vertex s, and to explore nothing twice, which suggests aiming for $O(m + n)$ time.

Algorithm 10.1: Generic graph search

Input: Graph $G = (V, E)$, undirected or directed, and a start vertex s.
Output: The vertices marked explored.

```
Mark s explored and every other vertex unexplored
while some edge (u, v) has u explored and v unexplored do
    Choose such an edge (u, v)
    Mark v explored
```

Proposition 10.1 [Search finds exactly the reachable vertices]. *For G undirected or directed, at the end of generic search a vertex v is explored if and only if G has a path from s to v.*

Proof. ($\Rightarrow$) Induct on the number of iterations. The vertex s is reachable, and a vertex v explored through an edge (u, v) with u already explored is reachable by extending an s–u path by that edge.
($\Leftarrow$) Suppose G has an s–v path P, but v is unexplored at the end. Since P starts at an explored vertex and ends at an unexplored one, some edge (u, x) of P has u explored and x unexplored. Then the loop condition still holds, so the algorithm would not have terminated, a contradiction. □

The generic procedure leaves open which frontier edge, crossing from explored to unexplored vertices, to choose. Two choices give the main search strategies.

- Breadth-first search (BFS) explores nodes in layers, using a queue (first in, first out), in $O(m + n)$ time. It computes shortest paths and the connected components of an undirected graph.

- Depth-first search (DFS) explores aggressively, like a maze, backtracking only when necessary. It uses a stack (last in, first out) or recursion, in $O(m + n)$ time. It computes a topological ordering of a directed acyclic graph and the strongly connected components of a directed graph.

10.2 Breadth-first search

Algorithm 10.2: Breadth-first search, BFS(G, s)

Input: Graph G with all vertices unexplored, and a start vertex s.
Output: The vertices reachable from s, marked explored.

```
1 Mark s explored
2 Q ← a queue containing only s
3 while Q ≠ ∅ do
4 |   Remove the first vertex v of Q
5 |   for each edge (v, w) do
6 |   |   if w is unexplored then
7 |   |   |   Mark w explored
8 |   |   |   Add w to the end of Q
```

The vertices at distance i from s form the layer L_i. BFS removes all of $L_0 = \{s\}$, then all of L_1, then all of L_2, and so on.

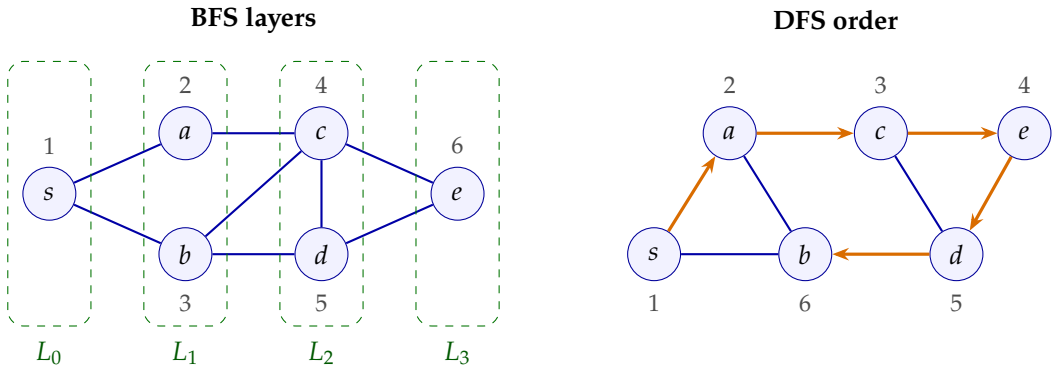


Figure 20: One undirected graph explored two ways. Left: BFS layers $L_0, \ldots, L_3$, with numbers giving the order in which vertices are explored. Right: DFS on a similar graph; the highlighted edges are the recursive calls, and the search backtracks only after reaching b.

Proposition 10.2 [Basic BFS properties]. *At the end of* BFS(G, s)*, a vertex v is explored if and only if G has an s–v path. The main loop runs in time $O(n_s + m_s)$, where n_s and m_s are the numbers of vertices and edges reachable from s.*

Proof. BFS is a special case of generic search: each newly explored w is reached by an edge from an explored v, and the loop stops only when no frontier edge remains. For the time bound, each reachable vertex enters Q at most once, because it is marked when added. Its

adjacency list is then scanned once, and each scanned edge costs $O(1)$. An edge is scanned at most twice (once from each end) in an undirected graph, and once in a directed graph. □

10.3 Shortest paths via BFS

Let $\text{dist}(v)$ be the fewest edges on an s–v path. Initialize $\text{dist}(s) = 0$ and $\text{dist}(v) = \infty$ for $v \neq s$. When BFS considers an edge (v, w) with w unexplored, it also sets $\text{dist}(w) \leftarrow \text{dist}(v) + 1$.

Proposition 10.3 [BFS computes shortest paths]. *At termination,* $\text{dist}(v) = i$ *if and only if* $v \in L_i$*, that is, a shortest* s–v *path has* i *edges.*

Proof. The idea is that every layer-i vertex w is added to Q by a layer-$(i-1)$ vertex v, via the edge (v, w). Prove by induction on i that the vertices of L_i enter Q after all vertices of earlier layers, before any vertex of a later layer, and with dist $= i$. This holds for $L_0 = \{s\}$. Assume it for layers up to $i - 1$. A vertex $w \in L_i$ has an edge from some vertex of L_{i-1}, and every vertex with an edge to w lies in L_{i-1} or a later layer. Hence w is still unexplored when the first such vertex of L_{i-1} is removed from Q, since vertices of later layers enter Q only after L_{i-1} does. At that moment w is added with $\text{dist}(w) = (i-1) + 1 = i$. Vertices of L_{i+1} and beyond have no edge from L_{i-1}, so they are added only after this. □

10.4 Undirected connectivity

Let G be undirected. Its connected components, the pieces of G, are the equivalence classes of the relation $u \sim v$ if and only if G has a u–v path. This is an equivalence relation: a single vertex is a path, reversing a path shows symmetry, and concatenating paths shows transitivity. Computing the components checks whether a network is disconnected and supports graph visualization and clustering.

Algorithm 10.3: Connected components via BFS

Input: Undirected graph G with vertices labeled $1, \ldots, n$.
Output: The connected components of G.

```
1 Mark every vertex unexplored
2 for i ← 1 to n do
3     if i is not yet explored (in some previous BFS) then
4         BFS(G, i)                    // discovers exactly the component of i
```

Each call discovers precisely the component of i, by Proposition 10.2, so every component is found once. Initialization and the outer loop take $O(n)$ time. Each BFS uses $O(1)$ time per vertex and per edge of its own component, and the components partition V and E. The total running time is therefore $O(m + n)$. For example, on a graph with vertex set $\{1, \ldots, 10\}$ whose components are

$\{1, 3, 5, 7, 9\}$, $\{2, 4\}$, and $\{6, 8, 10\}$, the loop calls BFS from 1, 2, and 6.

10.5 Depth-first search

DFS can mimic the BFS code with a stack in place of the queue, plus minor modifications. The recursive version is shorter.

Algorithm 10.4: Depth-first search, DFS(G, s)

Input: Graph G and a start vertex s.
Output: The vertices reachable from s, marked explored.

```
Mark s explored
for each edge (s, v) do
    if v is unexplored then
        DFS(G, v)
```

Proposition 10.4 [Basic DFS properties]. *At the end of* DFS(G, s)*, a vertex v is explored if and only if G has an s–v path, and the running time is $O(n_s + m_s)$.*

Proof. DFS is a particular instantiation of generic search. It makes at most one call per reachable vertex, and each call scans that vertex's edges once, so each edge is examined at most twice. □

10.6 Topological sort

Definition 10.5 [Topological ordering]. A topological ordering of a directed graph G is a labeling f of its vertices such that

1. the labels $f(v)$ are exactly $\{1, 2, \ldots, n\}$, and
2. $(u, v) \in E$ implies $f(u) < f(v)$.

The motivation is to sequence tasks while respecting all precedence constraints. If G has a directed cycle, it has no topological ordering: the labels would strictly increase around the cycle and return to their starting value. Conversely, if G is a directed acyclic graph (DAG), a topological ordering can be computed in $O(m + n)$ time. A DAG may have several topological orderings.

Lemma 10.6 [Sinks exist]. *Every directed acyclic graph has a sink vertex, one with no outgoing arcs.*

Proof. If no vertex were a sink, we could keep following outgoing arcs from any vertex. After n steps some vertex would repeat, giving a directed cycle. □

A straightforward algorithm follows: let v be a sink of G, set $f(v) = n$, and recurse on $G - \{v\}$, which is still acyclic. When v is assigned position i, all its outgoing arcs have already been deleted,

so they all lead to vertices later in the ordering.

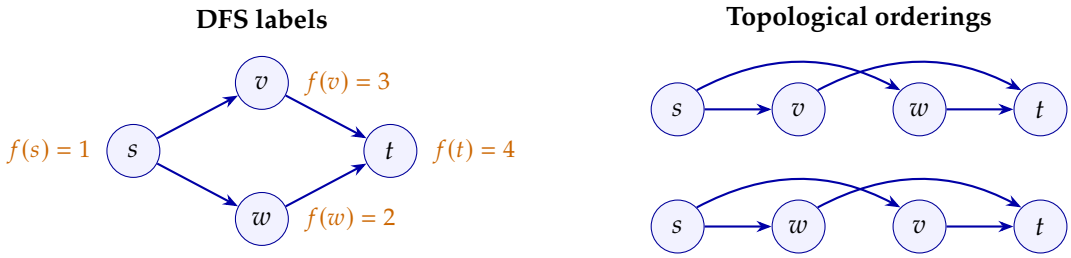


Figure 21: A DAG with edges $(s,v),(s,w),(v,t),(w,t)$. Left: the labels produced by the DFS method below, starting at s and exploring v before w. Right: its two topological orderings, with every arc pointing forward.

DFS gives a slicker linear-time method. Each vertex receives its label when its recursive call finishes, so labels are assigned in decreasing order.

Algorithm 10.5: Topological sort via DFS

```
Input: Directed acyclic graph G.
Output: A topological ordering f.
Mark every vertex unexplored
current_label ← n                                   // to keep track of the ordering
for each vertex v do
    if v is not yet explored (in a previous DFS call) then
        DFS(G, v)
procedure DFS(G, s):
    Mark s explored
    for each edge (s, v) do
        if v is not yet explored then
            DFS(G, v)
    f(s) ← current_label
    current_label ← current_label − 1
```

Theorem 10.7 [Topological sort via DFS]. *On a directed acyclic graph, this algorithm computes a topological ordering in $O(m+n)$ time.*

Proof. The running time is $O(1)$ per vertex and $O(1)$ per edge. Each vertex is labeled once, with labels $n, n-1, \ldots, 1$, so condition 1 holds. For condition 2, let (u,v) be an edge; we show $f(u) < f(v)$. Since labels decrease over time, it suffices that v's call finishes before u's.
Case 1: u is visited before v. When DFS(G,u) starts, v is unexplored, and the call scans the edge (u,v). So v is explored, and its recursive call finishes, before the call for u does. Hence $f(v) > f(u)$.
Case 2: v is visited before u. There is no path from v to u, since with the edge (u,v) it would close a directed cycle. So u is not explored during v's call, which finishes before u's call even starts. Hence $f(v) > f(u)$. □

10.7 Strongly connected components

Definition 10.8 [Strongly connected components]. The strongly connected components (SCCs) of a directed graph G are the equivalence classes of the relation $u \sim v$ if and only if G has both a path from u to v and a path from v to u.

The relation is reflexive and symmetric by definition and transitive by concatenating paths. A single DFS finds everything reachable from its start. Started in an SCC with no arcs leaving it, it discovers exactly that SCC; started elsewhere, it may discover the union of several SCCs. The order in which DFS calls are started therefore matters.

Theorem 10.9 [Kosaraju]. *The SCCs of a directed graph can be computed in $O(m + n)$ time.*

Kosaraju's two-pass algorithm uses one subroutine twice. With the vertices labeled $1, \ldots, n$, a global counter t (the number of vertices finished so far) records finishing times for the first pass, and a global vertex s (the current source) records leaders for the second.

Algorithm 10.6: DFS-Loop

```
Input: Directed graph G with vertices labeled 1, ..., n.
Output: Finishing times f(i) and leaders leader(i).
Mark every vertex unexplored; t ← 0; s ← NULL
for i ← n down to 1 do
    if i is not yet explored then
        s ← i
        DFS(G, i)
procedure DFS(G, i):
    Mark i explored; leader(i) ← s
    for each arc (i, j) ∈ E do
        if j is not yet explored then
            DFS(G, j)
    t ← t + 1; f(i) ← t                                  // i's finishing time
```

Algorithm 10.7: Kosaraju's two-pass algorithm

```
Input: Directed graph G.
Output: The SCCs of G.
G^rev ← G with every arc reversed
Run DFS-Loop on G^rev                                    // computes a good ordering f
Run DFS-Loop on G in decreasing order of f               // discovers the SCCs
return the groups of vertices with a common leader
```

The running time is that of two depth-first searches, $O(m + n)$. Building G^{rev} takes $O(m + n)$, and since the finishing times are a permutation of $1, \ldots, n$, relabeling each vertex by f orders the second pass without sorting.

In this example the finishing times of vertices $1, 2, \ldots, 9$ are $7, 3, 1, 8, 2, 5, 9, 4, 6$. The second pass

starts at vertex 7, the vertex with $f = 9$, and discovers exactly $\{1, 4, 7\}$.

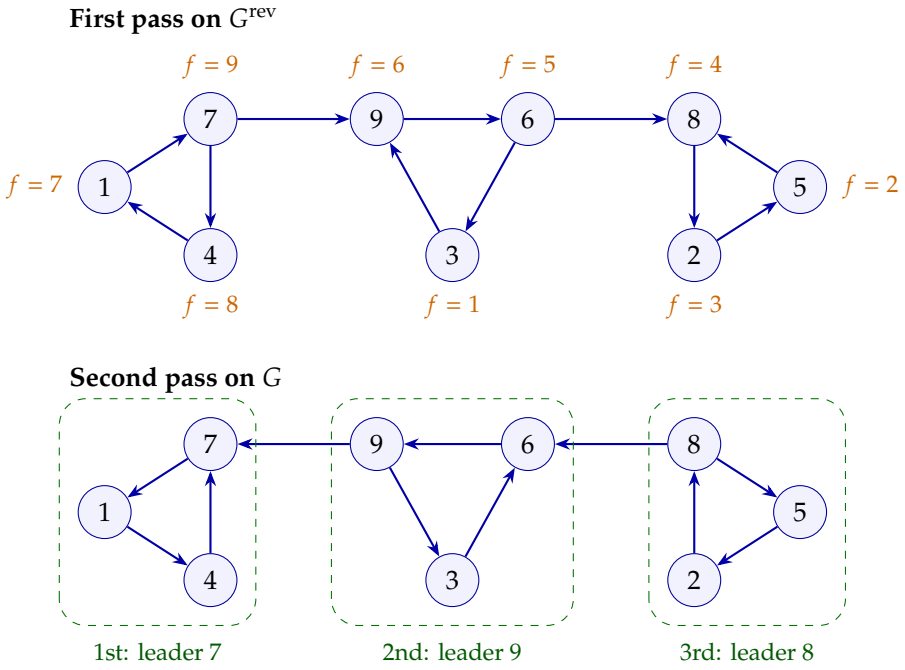


Figure 22: Kosaraju's algorithm on a nine-vertex graph. Top: the first pass on G^{rev}, scanning from vertex 9 down and, from 6, exploring 3 before 8; each vertex shows its finishing time. Bottom: the second pass on G in decreasing order of f discovers the three SCCs in turn, with leaders 7, 9, and 8 (finishing times 9, 6, 4).

10.8 Correctness of Kosaraju's algorithm

The SCCs induce an acyclic META-GRAPH: its nodes are the SCCs $C_1, \dots, C_k$ of G, with an arc $C \to \hat{C}$ if and only if G has an arc (i, j) with $i \in C$ and $j \in \hat{C}$. It is acyclic because a cycle of SCCs would collapse into a single SCC. Moreover, G and G^{rev} have exactly the same SCCs, since a u–v path in G is a v–u path in G^{rev}.

Lemma 10.10 [KEY LEMMA]. *Let C_1 and C_2 be SCCs of G joined by an arc of G from C_1 to C_2, and let f be the finishing times of DFS-Loop on G^{rev}. Then*

$$\max_{v \in C_1} f(v) < \max_{v \in C_2} f(v).$$

Proof. In G^{rev}, C_1 and C_2 are still SCCs, and the arc points from C_2 to C_1. Let v be the first vertex of $C_1 \cup C_2$ reached by the first pass.
Case 1: $v \in C_1$. There is no path from C_1 to C_2 in G^{rev}, since with the arc from C_2 to C_1 it would create a cycle in the meta-graph. So all of C_1 is explored and finished before C_2 is ever reached, and every f-value in C_1 is less than every f-value in C_2.
Case 2: $v \in C_2$. All of $C_1 \cup C_2$ is reachable from v and unexplored when v is reached, so

DFS(G^{rev}, v) does not finish until all of $C_1 \cup C_2$ has been completely explored. Thus $f(v) > f(w)$ for every $w \in C_1$. □

Corollary 10.11. *The maximum f-value lies in a* SINK SCC *of G, one with no arcs to other SCCs.*

Indeed, if the SCC containing the maximum had an arc to another SCC, that SCC would contain a larger f-value by Lemma 10.10.

Theorem 10.12 [CORRECTNESS OF KOSARAJU'S ALGORITHM]. *In the second pass, each call* DFS(G, i) *from the outer loop explores exactly one SCC, so vertices share a leader if and only if they lie in the same SCC.*

Proof. By Corollary 10.11, the second pass begins in a sink SCC C^*. The first call explores all of C^* and nothing else, since no arc leaves it. The rest of DFS-Loop then behaves like a run on G with C^* deleted.
In general, suppose the SCCs explored so far are each completely explored, and the next call starts at the unexplored vertex i with the largest f-value, lying in SCC C. If C had an arc to an unexplored SCC C', Lemma 10.10 would give a vertex of C' with f-value larger than $\max_{v \in C} f(v) \geq f(i)$, contradicting the choice of i. So every arc leaving C enters an explored SCC, and DFS(G, i) explores exactly C. By induction, successive calls peel off the SCCs one by one, in reverse topological order of the meta-graph. □

10.9 Structure of the Web

In the WEB GRAPH, vertices are Web pages and directed edges are hyperlinks. To ask what this graph looks like, assume it has already been crawled. Broder et al. (WWW 2000) computed the SCCs of a Web graph with roughly 200 million nodes and 1 billion edges, before MapReduce and Hadoop. They found a BOW TIE made of a giant SCC, the core of the Web; IN, pages that can reach the giant SCC but cannot be reached from it, such as new Web pages; OUT, pages reachable from the giant SCC that cannot reach back, such as corporate websites; tubes from IN to OUT that bypass the core; tendrils leaving IN or entering OUT; and disconnected pieces. The main findings were:

1. The four parts (giant SCC, IN, OUT, and tubes plus tendrils) have roughly the same size.
2. Within the core, the graph is very well connected: it has the SMALL WORLD property, in the sense of Milgram.
3. Outside the core, it is surprisingly poorly connected.

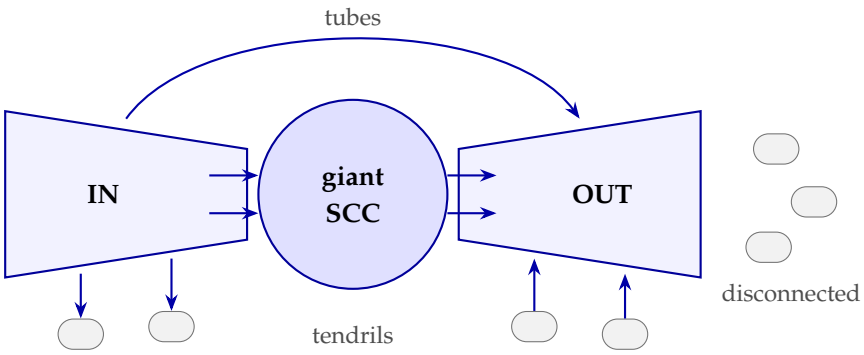


Figure 23: Schematic bow tie of the Web graph. Arrows show the direction in which each part can reach the others; sizes are not to scale.

Later research studies how the Web graph evolves over time, how new information propagates through the Web (or blogs, or Twitter), and how to define and compute communities in information and social networks.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Graph Search and Connectivity.

Lecture 11: Dijkstra's Shortest-Path Algorithm

11.1 Single-source shortest paths

Definition 11.1 [Single-source shortest paths]. The input is a directed graph $G = (V, E)$, with $m = |E|$ and $n = |V|$, a nonnegative length ℓ_e for each edge e, and a source vertex s. The length of a path is the sum of its edge lengths. The output is, for each $v \in V$, the length $L(v)$ of a shortest s–v path in G.

For convenience, assume that every $v \in V$ is reachable from s. The important assumption is $\ell_e \geq 0$ for every $e \in E$.

Breadth-first search already computes shortest paths in linear time, but only if $\ell_e = 1$ for every edge e. One could replace each edge e by a directed path of ℓ_e unit-length edges, but this blows up the graph too much. Dijkstra's algorithm handles general nonnegative lengths directly.

11.2 The algorithm

The algorithm grows a set X of processed vertices by one vertex per iteration. It records computed distances in A and computed shortest paths in B; the array B only helps the explanation and is not needed in an implementation.

Algorithm 11.1: Dijkstra's algorithm

Input: Directed graph $G = (V, E)$, lengths $\ell_e \geq 0$, source s.
Output: Distances $A[v]$ and paths $B[v]$ for every $v \in V$.

```
X ← {s}                                   // vertices processed so far
A[s] ← 0                                  // computed shortest-path distances
B[s] ← the empty path                     // computed shortest paths
while X ≠ V do
    Among all edges (v, w) ∈ E with v ∈ X and w ∉ X, choose the edge (v*, w*) minimizing
      A[v] + ℓ_vw
    Add w* to X
    A[w*] ← A[v*] + ℓ_v*w*
    B[w*] ← B[v*] ∪ (v*, w*)
```

The quantity $A[v] + \ell_{vw}$ is Dijkstra's greedy score of the edge (v, w); $A[v]$ was already computed in an earlier iteration. By the reachability assumption, some edge crosses from X to $V \setminus X$ while $X \neq V$, so each iteration can proceed.

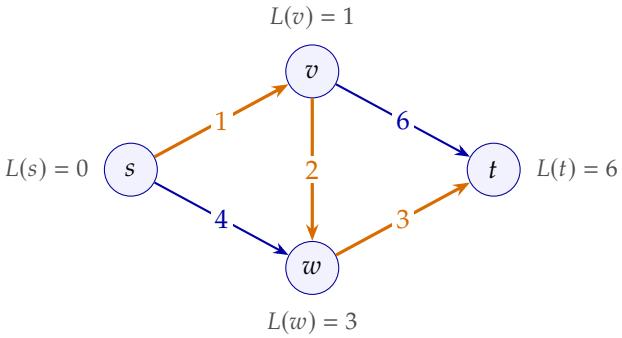


Figure 24: Running example. The shortest-path distances of s, v, w, t are $0, 1, 3, 6$; the highlighted edges form the shortest paths. Dijkstra's algorithm adds s, v, w, t to X in that order.

Example: In the running example, the first iteration compares the scores 1 of (s, v) and 4 of (s, w), and sets $A[v] = 1$ with $B[v] = s \to v$. The second compares 4 for (s, w), 3 for (v, w), and 7 for (v, t), and sets $A[w] = 3$ with $B[w] = s \to v \to w$. The third compares 7 for (v, t) and 6 for (w, t), and sets $A[t] = 6$ with $B[t] = s \to v \to w \to t$.

11.3 Negative edge lengths

One might try to reduce shortest paths with negative edge lengths to the nonnegative case by adding a large constant to every edge length. This does not preserve shortest paths, because paths with more edges are penalized more. Dijkstra's algorithm is also incorrect when lengths can be negative.

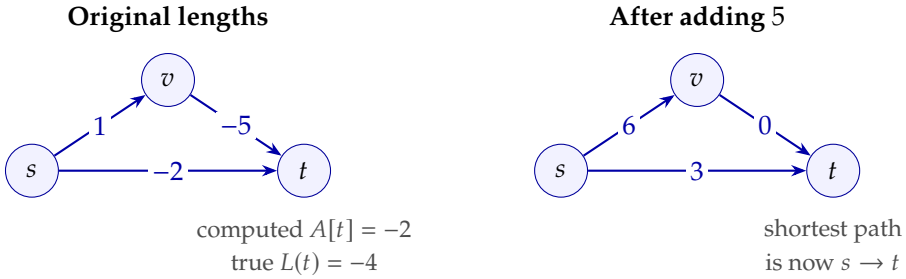


Figure 25: A graph with negative edge lengths. Dijkstra's algorithm computes the s–t distance as -2; the true distance is -4. Adding 5 to every length makes the direct edge, of length 3, shorter than the two-edge path, of length 6.

With $X = \{s\}$, the edge (s, t) has the smaller score -2, so t joins X with $A[t] = -2$ and $B[t] = s \to t$, although the path $s \to v \to t$ has length $1 - 5 = -4$.

11.4 Correctness

Theorem 11.2 [Dijkstra]. *For every directed graph with nonnegative edge lengths, Dijkstra's algorithm correctly computes all shortest-path distances: $A[v] = L(v)$ for every $v \in V$, where $A[v]$ is what the algorithm computes and $L(v)$ is the true shortest distance from s to v.*

Proof. Induct on the number of iterations. In the base case, $A[s] = L(s) = 0$ and $B[s]$ is the empty path.
Inductive hypothesis: all previous iterations were correct, that is, $A[v] = L(v)$ and $B[v]$ is a true shortest s–v path in G for every v already in X.
In the current iteration, the algorithm picks an edge (v^*, w^*) with $v^* \in X$ and $w^* \notin X$, and adds w^* to X. By the inductive hypothesis, $B[v^*]$ has length $L(v^*)$, so

$$A[w^*] = A[v^*] + \ell_{v^*w^*} = L(v^*) + \ell_{v^*w^*},$$

and $B[w^*] = B[v^*] \cup (v^*, w^*)$ is an s–w^* path of exactly this length. It remains to show that every s–w^* path has length at least $L(v^*) + \ell_{v^*w^*}$; then our path is a shortest one.
Let P be any s–w^* path. Since $s \in X$ and $w^* \notin X$, P must cross the frontier: it has a first edge (y, z) with $y \in X$ and $z \notin X$. So P consists of an s–y path, the edge (y, z), and a z–w^* path. Bound the three pieces:

- the s–y part has length at least $L(y) = A[y]$, by the inductive hypothesis since $y \in X$;
- the edge (y, z) has length ℓ_{yz};
- the z–w^* part has length at least 0, since no edge length is negative.

Thus P has length at least $A[y] + \ell_{yz}$. The edge (y, z) was a candidate in this iteration, so by Dijkstra's greedy criterion,

$$A[v^*] + \ell_{v^*w^*} \le A[y] + \ell_{yz} \le \text{length of } P.$$

The left side is the length of our path, which completes the induction. □

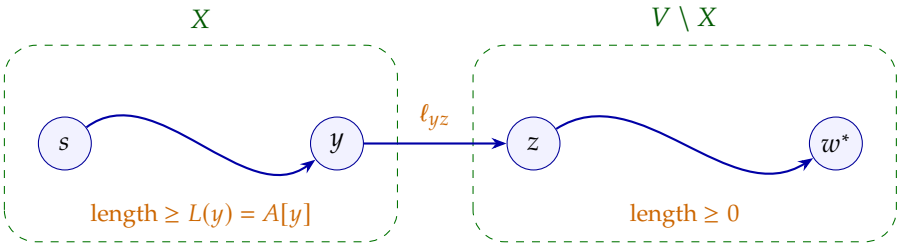


Figure 26: Any s–w^* path P leaves X through some first edge (y, z). Its three pieces have lengths at least $A[y]$, exactly ℓ_{yz}, and at least 0.

11.5 A heap-based implementation

A naive implementation runs $n - 1$ iterations of the main loop, each scanning all edges with $\Theta(1)$ work per edge, so $\Theta(m)$ work per iteration. Its running time is $\Theta(mn)$.

A HEAP performs Insert and Extract-Min in $O(\log n)$ time. Conceptually it is a perfectly balanced binary tree, of height about $\log_2 n$, satisfying the HEAP PROPERTY: at every node, the key is at most the children's keys. Extract-Min swaps the last leaf up to the root and bubbles it down; Insert adds a leaf and bubbles it up. Dijkstra's algorithm also needs to delete an element from the middle of the heap, which bubbles the replacement element up or down as needed, also in $O(\log n)$ time.

The implementation maintains two invariants.

1. The elements in the heap are the vertices of $V \setminus X$.
2. For each $v \notin X$, key$[v]$ is the smallest Dijkstra greedy score $A[u] + \ell_{uv}$ of an edge $(u, v) \in E$ with $u \in X$, or $+\infty$ if no such edge exists.

By the invariants, Extract-Min yields the correct vertex w^* to add to X next, and we set $A[w^*] \leftarrow$ key$[w^*]$: the minimum key is the minimum greedy score over all crossing edges.

When w moves from the heap to X, the edges leaving w may become new crossing edges, so some keys need updating.

Algorithm 11.2: Dijkstra's algorithm with a heap

Input: Directed graph $G = (V, E)$, lengths $\ell_e \geq 0$, source s.
Output: Distances $A[v]$ for every $v \in V$.

```
1  X ← {s}; A[s] ← 0
2  for each v ∈ V \ {s} do
3  |   key[v] ← min{ℓ_sv : (s, v) ∈ E}, or +∞ if there is no such edge
4  |   Insert v into the heap
5  while the heap is nonempty do
6  |   w ← Extract-Min; add w to X; A[w] ← key[w]
7  |   for each edge (w, v) ∈ E do
8  |   |   if v ∈ V \ X (that is, v is in the heap) then
9  |   |   |   Delete v from the heap
10 |   |   |   key[v] ← min{key[v], A[w] + ℓ_wv}        // greedy score of (w, v)
11 |   |   |   Re-insert v into the heap
```

The initial keys satisfy invariant 2 for $X = \{s\}$. After w joins X, the only new crossing edges into a vertex v of the heap are edges (w, v), and the key update accounts for exactly these. So both invariants hold after each iteration.

Theorem 11.3 [Running time]. *Dijkstra's algorithm with a heap runs in $O(m \log n)$ time.*

Proof. The running time is dominated by heap operations, each taking $O(\log n)$ time. There are $n-1$ initial insertions and $n-1$ Extract-Min operations. Each edge (v, w) triggers at most one Delete and Insert pair, when v is added to X first. So the number of heap operations is $O(n+m)$. Every vertex is reachable from s, so the graph is weakly connected and $m \geq n-1$; hence $O(n+m) = O(m)$. The total time is $O(m \log n)$, like sorting. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Dijkstra's Shortest-Path Algorithm.

Lecture 12: Heaps

A data structure organizes data so that it can be accessed quickly and usefully. Lists, stacks, queues, heaps, search trees, hash tables, Bloom filters, and union-find are examples. Different data structures support different sets of operations, so each suits different tasks. A good rule of thumb is to choose the minimal data structure that supports all the operations you need.

12.1 Heap operations

A heap is a container for objects that have keys, such as employee records, network edges, or events. With n the number of objects currently in the heap, it supports:

- *Insert*: add a new object, in $O(\log n)$ time;
- *Extract-Min*: remove an object with a minimum key, ties broken arbitrarily, in $O(\log n)$ time.

A heap can equally well support Extract-Max instead of Extract-Min. Two further operations are *Heapify*, which performs n batched inserts in $O(n)$ time, and *Delete*, which removes an arbitrary object in $O(\log n)$ time. A heap is also called a priority queue.

12.2 Applications

The canonical use of a heap is to perform repeated minimum computations quickly.

Sorting. Selection sort performs $\Theta(n)$ linear scans, each finding the minimum of the remaining elements, so it takes $\Theta(n^2)$ time on an array of length n. Replacing the scans by a heap gives heap sort.

Algorithm 12.1: Heap sort

Input: Array $A[1 : n]$.
Output: The entries of A in increasing order.

```
H ← an empty heap
for i ← 1 to n do
    Insert A[i] into H
for i ← 1 to n do
    B[i] ← ExtractMin(H)
return B
```

The running time is $2n$ heap operations, each $O(\log n)$, so $O(n \log n)$ in total. This is optimal for a comparison-based sorting algorithm.

Event manager. In a simulation, for example of a video game, the objects are event records, each an

action or update scheduled for a given future time. The key of an event is the time it is scheduled to occur, so Extract-Min yields the next scheduled event.

Median maintenance. Numbers $x_1, \dots, x_n$ arrive one by one. At each step i we must report the median of $\{x_1, \dots, x_i\}$, here the $\lceil i/2 \rceil$-th smallest, using $O(\log i)$ time per step. Maintain two heaps: H_{low} supporting Extract-Max and H_{high} supporting Extract-Min. The invariant is that H_{low} holds roughly the $i/2$ smallest elements and H_{high} the $i/2$ largest. Precisely, every element of H_{low} is at most every element of H_{high}, and the two sizes differ by at most one.

Algorithm 12.2: Median maintenance, step i

Input: Heaps H_{low} and H_{high} satisfying the invariant for $x_1, \dots, x_{i-1}$, and the new number x_i.
Output: The median of $x_1, \dots, x_i$.

```
if H_low is empty or x_i ≤ max H_low then
    Insert x_i into H_low
else
    Insert x_i into H_high
if |H_low| > |H_high| + 1 then
    Insert ExtractMax(H_low) into H_high
if |H_high| > |H_low| + 1 then
    Insert ExtractMin(H_high) into H_low
if |H_low| ≥ |H_high| then return max H_low
else return min H_high
```

The new element joins the correct side of the split, and moving the maximum of H_{low} (or the minimum of H_{high}) across keeps every low element below every high element, so the invariant is restored. If the two heaps have equal size, $\max H_{\text{low}}$ is the $(i/2)$-th smallest element. Otherwise the larger heap has $\lceil i/2 \rceil$ elements and its root is the $\lceil i/2 \rceil$-th smallest. Reading a root can be done by an extract followed by a re-insert. Each step uses a constant number of heap operations on at most i elements, hence $O(\log i)$ time.

Speeding up Dijkstra. For a graph with n vertices and m edges, the naive implementation of Dijkstra's shortest-path algorithm has n loop iterations, each a linear scan through the edges to compute a minimum, for a running time of $\Theta(nm)$. With heaps, the running time is $O(m \log n)$.

12.3 The heap property

Conceptually, a heap is a rooted binary tree that is as complete as possible: every level is full except possibly the last, which is filled from the left.

Definition 12.1 [Heap property]. At every node x, the key of x is at most the keys of the children of x.

Consequently, the object at the root has a minimum key. Indeed, following parent pointers from any node to the root never increases the key, so every key is at least the root's key. A given set of

keys usually has several valid heaps.

The tree needs no pointers. Store level 0 (the root), then level 1, level 2, and so on, each from left to right, in an array $A[1 : n]$. Level ℓ occupies positions 2^ℓ through $2^{\ell+1} - 1$, so

$$\text{parent}(i) = \lfloor i/2 \rfloor, \qquad \text{children}(i) = 2i,\ 2i + 1.$$

That is, the parent of i is $i/2$ if i is even and $(i - 1)/2$ if i is odd. Since every level but the last is full, a heap with n objects has $\lfloor \log_2 n \rfloor + 1$ levels, about $\log_2 n$.

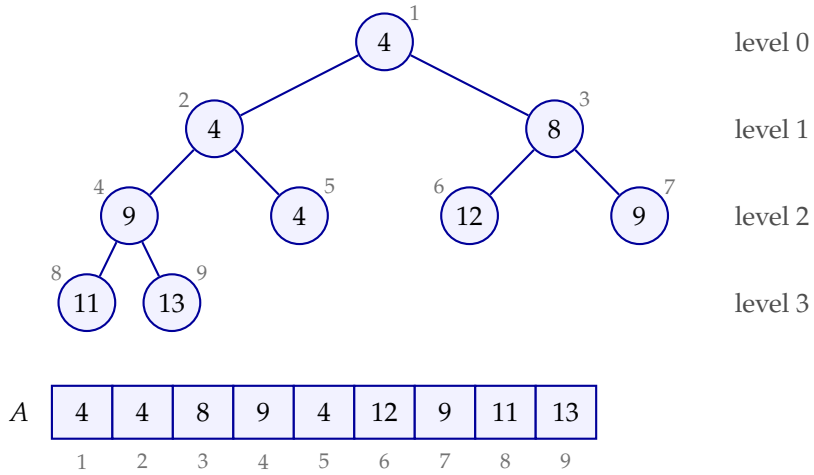


Figure 27: A heap with nine keys, drawn as a tree with each node's array position in gray, and stored as the array A. The children of position i are at positions $2i$ and $2i + 1$.

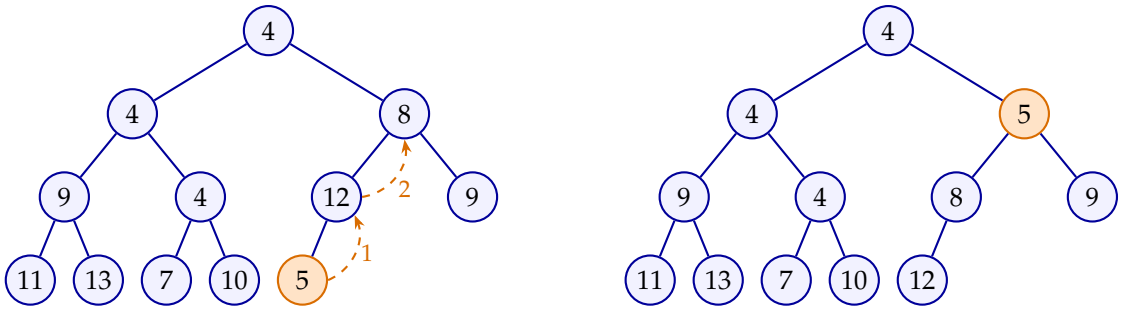
12.4 Insert and bubble-up

To insert a key k, place it at the end of the last level, that is, at position $n + 1$ of the array. This can break the heap property only between k and its parent. Bubble-up repeatedly swaps k with its parent until the parent's key is at most k, or k reaches the root.

Algorithm 12.3: Insert

Input: Heap $A[1 : n]$ and a new key k.
Output: Heap $A[1 : n + 1]$ containing k.

$n \leftarrow n + 1$; $A[n] \leftarrow k$; $i \leftarrow n$
while $i > 1$ **and** $A[\lfloor i/2 \rfloor] > A[i]$ **do**
    Swap $A[i]$ and $A[\lfloor i/2 \rfloor]$
    $i \leftarrow \lfloor i/2 \rfloor$



(a) 5 placed at the end

(b) after two swaps

Figure 28: Inserting 5 into the heap above, after 7 and 10 were inserted without any swaps. The new key swaps with 12, then with 8, and stops below the root, since $4 \le 5$.

Bubble-up stops, since k rises one level per swap. To see that it restores the heap property, let $x_0 \le x_1 \le \cdots \le x_{d-1}$ be the keys on the path from the root to the new leaf before the insertion. Bubble-up inserts k into this sorted sequence: each $x_i > k$ shifts down one node, and k fills the node vacated by the smallest of them, x_j. A shifted key x_i takes the place of x_{i+1}, so its new children are x_{i+1} and an off-path key that was already at least $x_{i+1} \ge x_i$. Similarly, k sits below $x_{j-1} \le k$ and above keys at least $x_j > k$. With about $\log_2 n$ levels and constant work per swap, Insert takes $O(\log n)$ time.

12.5 Extract-Min and bubble-down

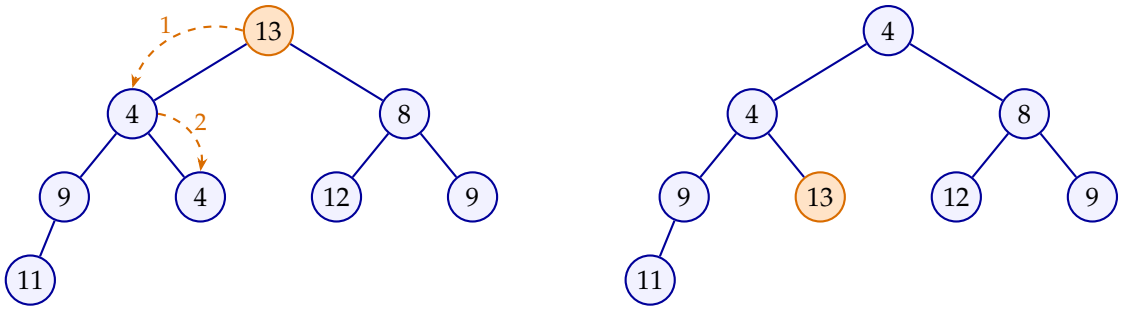
To extract the minimum, delete the root, move the last leaf to be the new root, and iteratively BUBBLE DOWN: swap the moved key with its *smaller* child until the heap property is restored.

Algorithm 12.4: Extract-Min

Input: Nonempty heap $A[1:n]$.
Output: A minimum key; $A[1:n-1]$ remains a heap.

```
m ← A[1]; A[1] ← A[n]; n ← n − 1; i ← 1
while 2i ≤ n do
    c ← 2i
    if 2i + 1 ≤ n and A[2i + 1] < A[2i] then c ← 2i + 1
    if A[i] ≤ A[c] then break
    Swap A[i] and A[c]; i ← c
return m
```

Swapping with the smaller child c is essential. After the swap, c's key sits above the moved key (larger than it) and above its former sibling (at least it), so the heap property holds at that node. Swapping with the larger child would place it above a smaller key. The only possible violation is between the moved key and its new children, one level lower. Hence there is at most one swap per level, the process halts with a heap, and Extract-Min takes $O(\log n)$ time.



(a) last leaf 13 moved to the root

(b) after two swaps

Figure 29: Extract-Min on the nine-key heap. After removing the root 4, the last leaf 13 moves to the root and swaps with its smaller child 4 twice, stopping at a leaf.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Heaps.

Lecture 13: Balanced Binary Search Trees

A sorted array supports many operations quickly, but not changes to the set of keys. A balanced search tree keeps those operations fast while also allowing logarithmic-time insertions and deletions.

13.1 Sorted arrays and balanced search trees

For a sorted array of n keys, *Select* returns the i-th order statistic, *Pred* and *Succ* take a pointer to a key and return its neighbors, and *Rank* returns the number of keys less than or equal to a given value. Inserting or deleting a key, however, takes $\Theta(n)$ time, since entries must shift. A balanced search tree is like a sorted array with fast inserts and deletes:

Operation	Sorted array	Balanced search tree
Search	$\Theta(\log n)$	$O(\log n)$
Select	$O(1)$	$O(\log n)$
Min / Max	$O(1)$	$O(\log n)$
Pred / Succ	$O(1)$	$O(\log n)$
Rank	$O(\log n)$	$O(\log n)$
Output in sorted order	$O(n)$	$O(n)$
Insert / Delete	$\Theta(n)$	$O(\log n)$

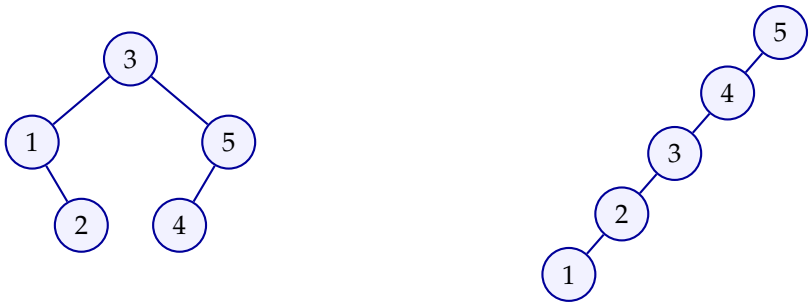
Select, Min/Max, and Pred/Succ become slower, from $O(1)$ to $O(\log n)$, in exchange for fast Insert and Delete. Hash tables also support Search, Insert, and Delete, and heaps support Insert, Delete, and Min (or Max); a search tree supports all of these operations at once.

13.2 Binary search tree structure

A binary search tree has exactly one node per key. In the most basic version, each node has a left child pointer, a right child pointer, and a parent pointer, any of which may be NULL.

Definition 13.1 [Search tree property]. At every node x, all keys in the left subtree of x are less than the key of x, and all keys in the right subtree of x are greater.

A set of keys has many possible search trees. The height, or depth, of a tree is the number of edges on a longest root-leaf path. For n keys it can be anywhere from about $\log_2 n$, when the tree is perfectly balanced, to $n - 1$, when it is a chain.



(a) height 2 (b) height 4

Figure 30: Two search trees on the keys $1, \ldots, 5$. Every key in a left subtree is smaller, and every key in a right subtree larger, than the key above it.

13.3 Search, insert, min, max, and predecessor

To *search* for a key k, start at the root and repeatedly follow the left child pointer if k is less than the current key, or the right child pointer if k is greater. Return the node with key k, or NULL if the search falls off the tree.

To *insert* a new key k, search for k, which fails, and rewire the final NULL pointer to a new node with key k. This preserves the search tree property: at each ancestor, the search went left exactly when k is smaller, so k lies on the correct side of every ancestor.

Each of these operations follows one root-leaf path, so its worst-case running time is $\Theta(\text{height})$. On a chain, this is $\Theta(n)$.

To find the *minimum* (*maximum*) key, start at the root and follow left (right) child pointers until this is no longer possible; the last key found is the answer.

To find the *predecessor* of a key k, the largest key smaller than k:

- if k's left subtree is nonempty, return the maximum key in it;
- otherwise, follow parent pointers until reaching a key less than k, which happens the first time the path turns left (moves up from a right child). If no such key exists, k is the minimum.

In the balanced tree of the figure above, the predecessor of 3 is 2, the maximum of its left subtree, and the predecessor of 4 is 3, reached by moving up through 5. The successor is symmetric.

Proposition 13.2 [Predecessor rule]. *Both cases of this procedure return the predecessor of k.*

Proof. First, a node v outside the subtree of a node u is either smaller than every key in that subtree or larger than all of them: at the lowest common ancestor of u and v, the two lie on

different sides, or v is that ancestor. So in the first case, a key strictly between the maximum of k's left subtree and k would lie in k's subtree; but the left subtree has no larger key and the right subtree no smaller one. In the second case, let a be the first ancestor with a smaller key. Below a, the path to k starts at a's right child and then makes only left moves, and k has no left child, so k is the minimum of a's right subtree. By the mirror image of the first case, that minimum is the successor of a; hence a is the predecessor of k. □

13.4 In-order traversal and deletion

Algorithm 13.1: In-order traversal

Input: The root r of a search tree, with subtrees T_L and T_R.
Output: The keys of the tree, printed in increasing order.

```
1 if r = NULL then return
2 Recurse on T_L
3 Print the key of r
4 Recurse on T_R
```

By induction, the recursive calls print the keys of T_L, all smaller than r's key, and the keys of T_R, all larger, in increasing order. There is $O(1)$ work per call and one call per node (plus NULL children), so the traversal takes $O(n)$ time.

To *delete* a key k, first search for it. Then:

- *Easy case* (k's node has no children): remove the node.
- *Medium case* (k's node has one child): splice out the node; its unique child takes the position previously held by k's node.
- *Difficult case* (k's node has two children): compute k's predecessor ℓ by following k's left child pointer and then right child pointers until no longer possible. Swap k and ℓ. In its new position k has no right child, so delete or splice it out by the easy or medium case.

The result is a valid search tree. In the difficult case, ℓ is the largest key in k's left subtree, so in k's old position it exceeds every remaining key of the left subtree and, being less than k, is below every key of the right subtree. For example, in the balanced tree above, deleting 2 is the easy case, deleting 5 moves 4 into its place, and deleting 3 swaps it with 2 and then removes a leaf. Deletion takes Θ(height) time.

13.5 Select and rank

The idea is to store a little extra information at each node about the tree itself, not about the data. A useful augmentation is size(x), the number of nodes in the subtree rooted at x. If x has children y

and z, then size(x) = size(y) + size(z) + 1, counting the left subtree, the right subtree, and x itself (a missing child has size 0). Sizes are easy to keep up to date during insertion and deletion, since only the nodes on the affected root-leaf path change.

Algorithm 13.2: Select the i-th order statistic

Input: Root x of a size-augmented search tree, and $1 \le i \le$ size(x).
Output: The i-th smallest key in the subtree of x.

```
1 Let y and z be the left and right children of x
2 a ← size(y)                                   // a = 0 if x has no left child
3 if a = i − 1 then return the key of x
4 if a ≥ i then return Select(y, i)
5 return Select(z, i − a − 1)
```

The left subtree holds the a smallest keys and the right subtree the larger ones, so the answer is x, in the left subtree, or in the right subtree with $a + 1$ smaller keys skipped. For example, in the balanced tree above, the 4th smallest key goes right at 3 (with $a = 2$), asks for the 1st smallest under 5, goes left (with $a = 1$), and returns 4. Select takes Θ(height) time.

Rank is similar. Search for the given value; each time the search moves right from a node x, or finds the value at x, add size(left child of x) + 1: these are x and the keys below it that are smaller. The total counts exactly the keys at most the value, also in Θ(height) time.

13.6 Red-black trees

A BALANCED SEARCH TREE ensures that its height is always $O(\log n)$, which is best possible. Then Search, Insert, Delete, Min, Max, Pred, and Succ all run in $O(\log n)$ time. Red-black trees (Bayer 1972; Guibas and Sedgewick 1978) are one example; AVL trees, splay trees, and B-trees are others.

Definition 13.3 [RED-BLACK TREE]. A RED-BLACK TREE is a binary search tree satisfying four invariants:

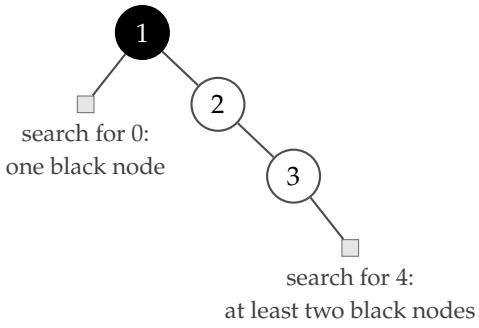
(1) each node is red or black;

(2) the root is black;

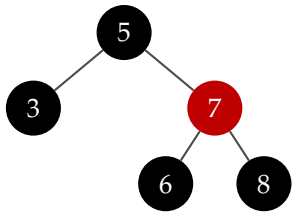
(3) there are no two reds in a row: a red node has only black children;

(4) every root-NULL path, as in an unsuccessful search, has the same number of black nodes.

Example: A chain of three keys 1, 2, 3 cannot be a red-black tree. The root 1 is black, so an unsuccessful search for 0 passes exactly one black node. An unsuccessful search for 4 passes 1, 2, and 3; since 2 and 3 cannot both be red, it passes at least two black nodes, violating invariant (4). By contrast, the tree with root 5, children 3 and 7, and 7's children 6 and 8 is a red-black tree when 7 is red and the rest are black: every root-NULL path has two black nodes.



(a) a chain of three keys



(b) a red-black tree

Figure 31: (a) The two unsuccessful searches, ending at the gray NULL pointers, pass different numbers of black nodes however 2 and 3 are colored. (b) A valid coloring: every root-NULL path has two black nodes.

Theorem 13.4 [HEIGHT GUARANTEE]. *Every red-black tree with n nodes has height at most $2\log_2(n+1)$.*

Proof. Let k be the minimum number of nodes on a root-NULL path. Then the top k levels of the tree are full: it contains a perfectly balanced search tree of depth $k-1$. Hence

$$n \geq 2^k - 1, \qquad \text{so} \qquad k \leq \log_2(n+1).$$

Thus some root-NULL path has at most $\log_2(n+1)$ nodes, and in particular at most $\log_2(n+1)$ black nodes. By invariant (4), every root-NULL path has at most $\log_2(n+1)$ black nodes. By invariants (2) and (3), each red node on a path has a black parent on the same path, so reds are at most as many as blacks. Every root-NULL path therefore has at most $2\log_2(n+1)$ nodes, which bounds the height. □

Search, Min, Max, Pred, Succ, Select, and Rank do not modify the tree, so they run unchanged in $O(\log n)$ time. Insert and Delete must be reimplemented so that the invariants are maintained.

13.7 Rotations

The key primitive, common to all balanced search tree implementations (red-black, AVL, B+, and others), is the ROTATION: it locally rebalances the subtrees at a node in $O(1)$ time. A left rotation of a parent x and its right child y (so $x < y$) makes y the child of x's former parent p and makes x the left child of y. The left subtree A of x stays with x, the right subtree C of y stays with y, and the subtree B of keys between x and y moves from y's left to x's right.

A right rotation of a parent and its left child is the mirror image. Both maintain the search tree property, since the in-order sequence A, x, B, y, C is unchanged, and both rewire a constant number of pointers, so they take $O(1)$ time.

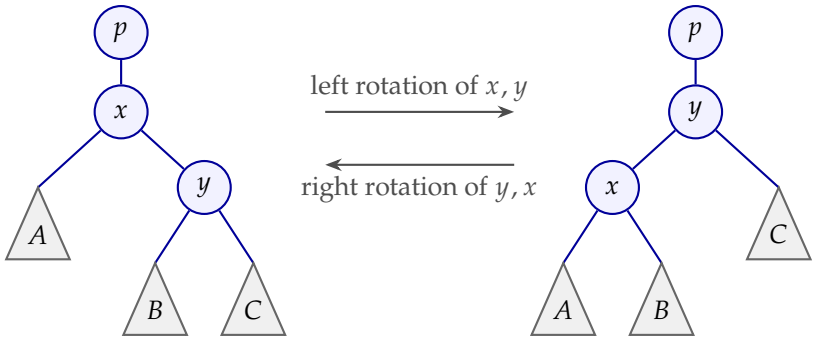


Figure 32: A left rotation and its inverse right rotation. Subtree A holds the keys less than x, B the keys between x and y, and C the keys greater than y.

13.8 Insertion into a red-black tree

The idea for Insert and Delete is to proceed as in an ordinary binary search tree, then recolor and perform rotations until the invariants are restored. To insert x:

1. Insert x as usual, which makes x a leaf.
2. Color x red. This preserves invariant (4), since it adds no black node to any root-NULL path.
3. If x's parent y is black, all invariants hold; done.
4. Otherwise y is red, a double red violating invariant (3). Then y is not the root, and its parent w is black.

In general, let x, y be the current double red with x the deeper node, and let w be x's grandparent, which is black. Let z be w's other child, if any.

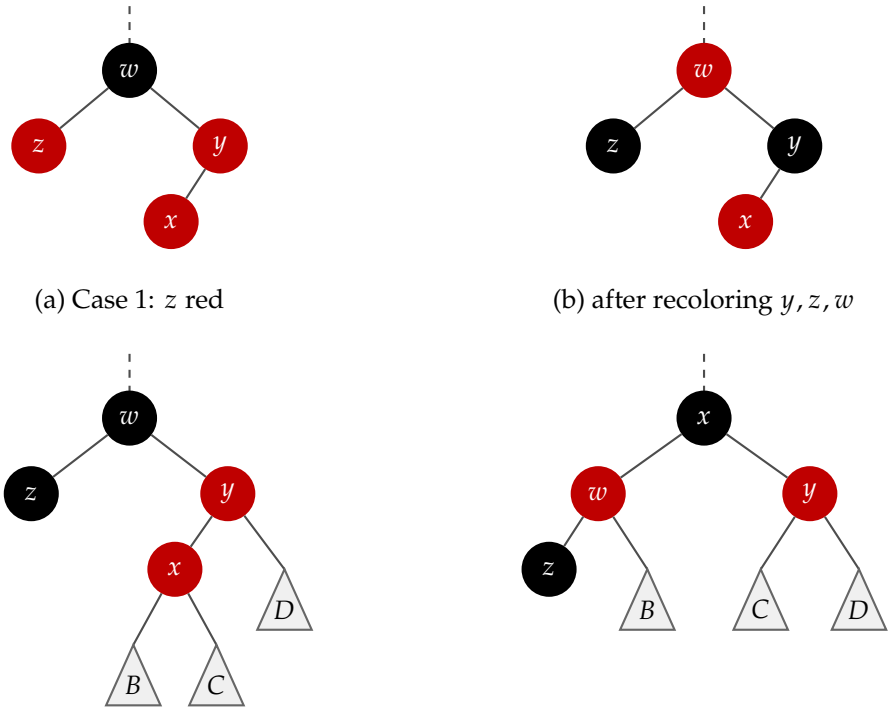
Case 1: z is red. Recolor y and z black and w red. Every root-NULL path through w loses the black w and gains exactly one of y and z, so invariant (4) still holds. Either invariant (3) is restored, or w and its parent form a new double red, two levels higher. This can happen only $O(\log n)$ times. If the double red reaches the root, recolor the root black, which adds one black node to every root-NULL path and so preserves invariant (4).

Case 2: z is black or NULL. The double red can be eliminated in $O(1)$ time by rotations and recolorings, after which all invariants are satisfied. Let B, C, and D be the remaining subtrees below x and y, as in the figure. Rearrange w, y, x so that the middle key of the three is on top, colored black, with the other two as its red children:

- if x and y are both left or both right children, rotate w and y, putting y on top;
- otherwise, as in the figure, first rotate y and x, which reduces to the previous situation with x in place of y, then rotate w and x.

The new top is black and its children are red, while every subtree that hangs below them has a

black root or is empty: z is black or NULL, and the children of the red nodes x and y were black or NULL. So there is no double red, and every root-NULL path passes the same number of black nodes as before, one at the top. If w was the root, the new top is a black root.



(a) Case 1: z red

(b) after recoloring y, z, w

(c) Case 2: z black or NULL

(d) after two rotations and recoloring

Figure 33: Fixing a double red x, y with grandparent w and uncle z. Case 1 recolors and may push the double red up to w and its parent. Case 2 restructures with x in the middle on top, keeping the in-order sequence z, w, B, x, C, y, D.

Since Case 1 repeats at most $O(\log n)$ times and Case 2 occurs at most once, insertion takes $O(\log n)$ time.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Balanced Binary Search Trees.

Lecture 14: Hashing: The Basics

A hash table maintains a possibly evolving set of records, such as transactions, people with associated data, or IP addresses. Each record is identified by a key, and the table supports three operations:

- *Insert*: add a new record;
- *Delete*: remove an existing record;
- *Lookup*: check for a particular record.

A structure supporting these operations is a dictionary. A hash table performs all three in $O(1)$ time, provided it is properly implemented and the data are not pathological. Both caveats are explained below and in Lecture 15.

14.1 Applications

Hash tables are useful whenever an algorithm repeatedly asks whether it has seen something before.

Example: De-duplication. Objects arrive in a stream, from a linear scan through a huge file or in real time. The goal is to keep track of the unique objects, for example to report unique visitors to a web site or to avoid duplicates in search results.

Algorithm 14.1: De-duplication with a hash table

```
Input: A stream of objects.
Output: The distinct objects of the stream, stored in H.
H ← an empty hash table
for each arriving object x do
    if Lookup(H, x) fails then
        Insert(H, x)
```

Each arrival costs one Lookup and at most one Insert, so the work per object is constant.

Definition 14.1 [2-SUM]**.** Given an unsorted array A of n integers and a target sum t, determine whether A contains two numbers x, y with $x + y = t$.

Exhaustive search over all pairs takes $\Theta(n^2)$ time. Sorting A in $\Theta(n \log n)$ time and then binary searching for $t - x$ for each x in A takes another $\Theta(n \log n)$ time. A hash table removes the logarithmic factors.

Algorithm 14.2: 2-SUM with a hash table

Input: Array $A[1:n]$ of integers and target t.
Output: Whether some x, y in A satisfy $x + y = t$.

```
H ← an empty hash table
for i ← 1 to n do
    Insert(H, A[i])
for i ← 1 to n do
    if Lookup(H, t − A[i]) succeeds then
        return true
return false
```

Both loops take $\Theta(n)$ time. If x and y must occupy different positions, a match with $t - x = x$ also requires x to occur twice in A, which a count stored with each key detects.

Further applications include symbol tables in compilers (the historical application), blocking network traffic, and search algorithms such as game tree exploration. In the last case, a hash table of visited configurations, such as arrangements of chess pieces, prevents exploring any configuration more than once.

14.2 Buckets and hash functions

Let U be a UNIVERSE of possible keys, such as all IP addresses, all names, or all chessboard configurations. It is generally very large. The goal is to maintain an evolving set $S \subseteq U$, which is generally of reasonable size.

There are two naive solutions. An array indexed by the elements of U supports $O(1)$ operations but uses $\Theta(|U|)$ space. A list of the elements of S uses $\Theta(|S|)$ space but takes $\Theta(|S|)$ time per Lookup. A hash table achieves the space of the list and the speed of the array:

1. pick a number n of BUCKETS with n comparable to $|S|$, assuming for simplicity that $|S|$ does not vary much;
2. choose a HASH FUNCTION $h : U \to \{0, 1, \ldots, n-1\}$;
3. use an array A of length n, and store x in $A[h(x)]$.

Definition 14.2 [COLLISION]. A COLLISION is a pair of distinct keys $x, y \in U$ with $h(x) = h(y)$.

Collisions are unavoidable in practice, even when keys are assigned to buckets uniformly at random. The BIRTHDAY PARADOX shows this with days of the year as buckets and people with uniformly random birthdays.

The probability that two of them share a birthday is

Number of people	Probability of a shared birthday
23	at least 50%
57	about 99%
184	99.99 . . . %
367	100%

The last row is the pigeonhole principle, allowing for February 29. Thus a collision is likely long before the number of objects approaches the number of buckets.

14.3 Resolving collisions

There are two standard ways to store colliding keys.

Definition 14.3 [Chaining]. In separate chaining, each bucket holds a linked list. Insert, Delete, and Lookup for a key x are performed in the list stored in $A[h(x)]$.

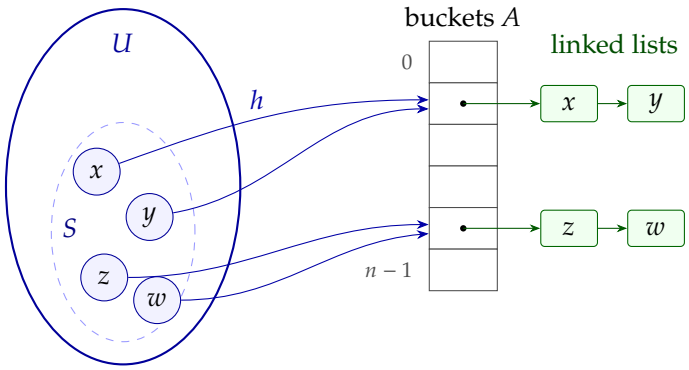


Figure 34: Separate chaining. The hash function sends each key of S to a bucket of A; colliding keys share the linked list of their bucket.

Definition 14.4 [Open addressing]. In open addressing, each bucket holds at most one object. The hash function specifies a probe sequence $h_1(x), h_2(x), \ldots$ for each key, and an operation tries these buckets in order until it finds an open slot (for Insert) or the key (for Lookup).

Linear probing looks at consecutive buckets after the first probe. Double hashing uses two hash functions, one to choose the first bucket and one to choose the step between probes.

14.4 What makes a good hash function

With chaining, Insert takes $\Theta(1)$ time, since a new object goes at the front of the list in $A[h(x)]$. Lookup and Delete take time proportional to the length of that list. For m objects, the longest list

could be anywhere from m/n, when all lists have equal length, to m, when all objects land in the same bucket. Performance therefore depends on the choice of hash function; open addressing behaves analogously.

A good hash function should

1. lead to good performance, that is, spread the data out; the gold standard is completely random hashing;
2. be easy to store and very fast to evaluate.

Example: BAD HASH FUNCTIONS. Take ten-digit phone numbers as keys, so $|U| = 10^{10}$, and choose $n = 10^3$ buckets. Using the first three digits (the area code) is terrible, since many keys share an area code. Using the last three digits is mediocre: it is still vulnerable to patterns in those digits. For memory locations as keys, which are multiples of a power of 2, the function $h(x) = x \bmod 1000$ (again $n = 10^3$) leaves every odd bucket empty, because an even x has an even remainder modulo 1000.

A quick-and-dirty hash function composes two maps. A HASH CODE converts an object to an integer, for example a subroutine converting strings to integers. A COMPRESSION FUNCTION, such as $z \mapsto z \bmod n$, then maps the integer to a bucket.

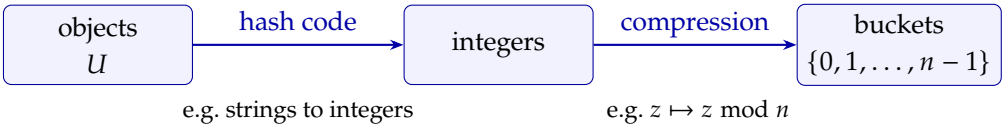


Figure 35: A quick-and-dirty hash function in two stages: a hash code maps objects to integers, and a compression function maps integers to buckets.

The number of buckets n should be chosen

1. prime, and within a constant factor of the number of objects in the table;
2. not too close to a power of 2;
3. not too close to a power of 10.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Hashing: The Basics.

Lecture 15: Universal Hashing

A hash table gives constant-time operations only when its load is controlled and its hash function spreads the data out. No single hash function spreads every data set; choosing the function at random from a suitable family does so for every data set in expectation.

15.1 The load of a hash table

Definition 15.1 [Load factor]. The load factor of a hash table is

$$\alpha = \frac{\text{number of objects in the table}}{\text{number of buckets}}.$$

Only chaining remains feasible for $\alpha > 1$: open addressing stores at most one object per bucket, so it requires $\alpha \leq 1$. Two further observations follow.

1. $\alpha = O(1)$ is necessary for operations to run in constant time. With chaining, some list has length at least α.
2. With open addressing, one needs $\alpha \ll 1$.

Good performance therefore requires controlling the load. Deletion is also easier and more common with chaining than with open addressing.

15.2 Pathological data sets

Good performance also requires a good hash function. Ideally, a single clever hash function would spread every data set out evenly. No such function exists.

Proposition 15.2 [Every hash function has a pathological data set]. *For every hash function $h : U \to \{0, 1, \ldots, n-1\}$, some bucket i receives at least $|U|/n$ elements of U.*

Proof. The n buckets partition U into the preimages $h^{-1}(0), \ldots, h^{-1}(n-1)$, whose sizes sum to $|U|$. By the pigeonhole principle, at least one has size at least $|U|/n$. A data set drawn only from that preimage has every object in the same bucket, so every pair collides. □

Since U is generally much larger than n, such a data set can be as large as desired. Pathological data arise in practice: Crosby and Wallach (USENIX 2003) showed that several real-world systems, such as network intrusion detection, can be paralyzed by exploiting badly designed hash functions.

Their targets were open source and used overly simplistic hash functions, so pathological data sets were easy to reverse engineer.

There are two remedies.

1. Use a cryptographic hash function, such as SHA-2, for which reverse engineering a pathological data set is infeasible.
2. Use randomization: design a family $\mathcal{H}$ of hash functions such that for every data set S, almost all functions $h \in \mathcal{H}$ spread S out pretty evenly.

The second remedy resembles the QuickSort guarantee: the randomness lies in the algorithm, and the guarantee holds for every input. The rest of this lecture develops it.

15.3 Universal families of hash functions

Definition 15.3 [Universal family]. Let $\mathcal{H}$ be a set of hash functions from U to $\{0, 1, \ldots, n-1\}$, where n is the number of buckets. Then $\mathcal{H}$ is universal if for all distinct $x, y \in U$,

$$\mathbb{P}_{h \in \mathcal{H}}\big[h(x) = h(y)\big] \leq \frac{1}{n},$$

where h is chosen uniformly at random from $\mathcal{H}$.

The collision probability of each pair is thus as small as with the gold standard of perfectly random hashing. A weaker per-key condition does not suffice.

Example: Suppose that for every bucket i and key k, exactly a $1/n$ fraction of the functions in $\mathcal{H}$ map k to i. Then $\mathcal{H}$ may or may not be universal. The family of all functions from U to $\{0, 1, \ldots, n-1\}$ has this property and is universal. The family of the n constant functions also has it, but every pair of keys collides with probability 1.

15.4 A universal family for IP addresses

Let U be the set of IP addresses $x = (x_1, x_2, x_3, x_4)$ with each $x_i \in \{0, 1, \ldots, 255\}$. Let n be a prime, for example a small multiple of the number of objects in the table, with $n > 255$. For each $a = (a_1, a_2, a_3, a_4)$ with every $a_i \in \{0, 1, \ldots, n-1\}$, define

$$h_a(x_1, x_2, x_3, x_4) = \big(a_1x_1 + a_2x_2 + a_3x_3 + a_4x_4\big) \bmod n,$$

and let $\mathcal{H} = \{h_a : a \in \{0, 1, \ldots, n-1\}^4\}$. There are n^4 such functions. Each is stored as four numbers and evaluated with four multiplications.

Theorem 15.4 [Universality of the IP-address family]. *The family $\mathcal{H}$ is universal.*

Proof. Let x and y be distinct IP addresses. They differ in some coordinate; by symmetry, assume $x_4 \neq y_4$. Rearranging, x and y collide under h_a if and only if

$$a_4(x_4 - y_4) \equiv \sum_{i=1}^{3} a_i(y_i - x_i) \pmod{n}. \tag{15.1}$$

Condition on arbitrary values of a_1, a_2, a_3, so the right-hand side is some fixed number in $\{0, 1, \ldots, n-1\}$, while a_4 is still uniformly random.
Key claim: the left-hand side is equally likely to be any of $0, 1, \ldots, n-1$. Write $d = x_4 - y_4$. Since $0 < |d| \leq 255 < n$, we have $d \not\equiv 0 \pmod{n}$. If $a_4 d \equiv a_4' d \pmod{n}$, then n divides $(a_4 - a_4')d$; as n is prime and does not divide d, it divides $a_4 - a_4'$, so $a_4 = a_4'$. Hence $a_4 \mapsto a_4 d \bmod n$ is a bijection of $\{0, 1, \ldots, n-1\}$, and a uniform a_4 gives a uniform left-hand side.
Exactly one value of a_4 therefore satisfies equation (15.1), so the conditional collision probability is $1/n$. This holds for every choice of a_1, a_2, a_3, hence $\mathbb{P}[h_a(x) = h_a(y)] = 1/n$. □

Example: With $n = 7$, the multiples $a_4 d \bmod 7$ for $a_4 = 0, 1, \ldots, 6$ are

a_4	0	1	2	3	4	5	6
$2a_4 \bmod 7$	0	2	4	6	1	3	5
$3a_4 \bmod 7$	0	3	6	2	5	1	4

Each residue appears exactly once in each row.

15.5 Chaining with a universal family

Theorem 15.5 [Carter and Wegman, 1979]. *Implement a hash table with chaining, and choose its hash function h uniformly at random from a universal family $\mathcal{H}$. Then for every data set S, all operations run in $O(1)$ expected time, assuming*

1. *the expectation is over the random choice of h;*
2. *$|S| = O(n)$, that is, the load $\alpha = |S|/n$ is $O(1)$;*
3. *evaluating h takes $O(1)$ time.*

Proof. Analyze an unsuccessful Lookup; the other operations are only faster. Let S be an arbitrary data set with $|S| = O(n)$, and consider a Lookup for $x \notin S$. Its running time is $O(1)$ to compute $h(x)$ plus $O(L)$ to traverse the list, where L is the length of the list in $A[h(x)]$. Here L is a random variable depending on h.
Use the decomposition principle: express the random variable as a sum of indicator variables,

then apply linearity of expectation. For each $y \in S$, so $y \neq x$, define

$$Z_y = \begin{cases} 1 & \text{if } h(y) = h(x), \\ 0 & \text{otherwise.} \end{cases}$$

Then $L = \sum_{y \in S} Z_y$, and $\mathbb{E}[Z_y] = 0 \cdot \mathbb{P}[Z_y = 0] + 1 \cdot \mathbb{P}[Z_y = 1] = \mathbb{P}[h(y) = h(x)]$. Since $\mathcal{H}$ is universal, each such probability is at most $1/n$. Therefore

$$\mathbb{E}[L] = \sum_{y \in S} \mathbb{E}[Z_y] = \sum_{y \in S} \mathbb{P}[h(y) = h(x)] \leq \sum_{y \in S} \frac{1}{n} = \frac{|S|}{n} = \alpha = O(1).$$

The expected running time is $O(1) + O(\mathbb{E}[L]) = O(1)$. □

15.6 Open addressing: heuristic analysis

With open addressing, each slot holds one object and the hash function produces a probe sequence for each key. A rigorous analysis is difficult. For a quick idealized analysis, make the HEURISTIC ASSUMPTION that all $n!$ probe sequences are equally likely.

Proposition 15.6 [INSERTION UNDER THE HEURISTIC ASSUMPTION]. *Under the heuristic assumption, the expected insertion time is about $1/(1-\alpha)$, where α is the load.*

Proof. A random probe finds an empty slot with probability about $1 - \alpha$. So the insertion time is about the number N of flips of a coin needed to get heads, where $\mathbb{P}[\text{heads}] = 1 - \alpha$. The first flip always happens; with probability α it is tails, and the expected number of further flips is again $\mathbb{E}[N]$. Thus

$$\mathbb{E}[N] = 1 + \alpha\,\mathbb{E}[N], \qquad \text{so} \qquad \mathbb{E}[N] = \frac{1}{1-\alpha}.$$ □

The bound grows without limit as $\alpha \to 1$, which is why open addressing needs $\alpha \ll 1$.

For linear probing, the heuristic assumption is completely false. A less false assumption is that the initial probes are uniformly random and independent for different keys.

Theorem 15.7 [KNUTH, 1962]. *Under this assumption, the expected insertion time with linear probing is about*

$$\frac{1}{(1-\alpha)^2},$$

where α is the load.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Universal Hashing.

Lecture 16: Bloom Filters

A Bloom filter supports fast Inserts and Lookups for a set of keys. Compared with a hash table, it is more space efficient, but it

1. cannot store an object associated with a key;
2. does not support deletions;
3. has a small false positive probability: Lookup may report that x has been inserted even though it has not.

Bloom filters were originally used in early spellcheckers. A canonical application is a list of forbidden passwords. A modern application is in network routers, which have limited memory and need to be very fast.

16.1 The data structure

A Bloom filter consists of

1. an array A of n bits, all initially 0;
2. k hash functions $h_1, \ldots, h_k$, where k is a small constant.

For a data set S, the quantity $b = n/|S|$ is the number of bits per object.

Algorithm 16.1: Bloom filter operations

```
1 procedure Insert(x)
2     for i ← 1 to k do
3         A[h_i(x)] ← 1                    // whether or not the bit is already 1
4 procedure Lookup(x)
5     return true if A[h_i(x)] = 1 for every i ∈ {1, ..., k}, and false otherwise
```

There are no false negatives: if x was inserted, its k bits were set to 1 and are never reset, so Lookup(x) succeeds. A false positive occurs when x was not inserted but all k bits $A[h_i(x)]$ were already set to 1 by other insertions.

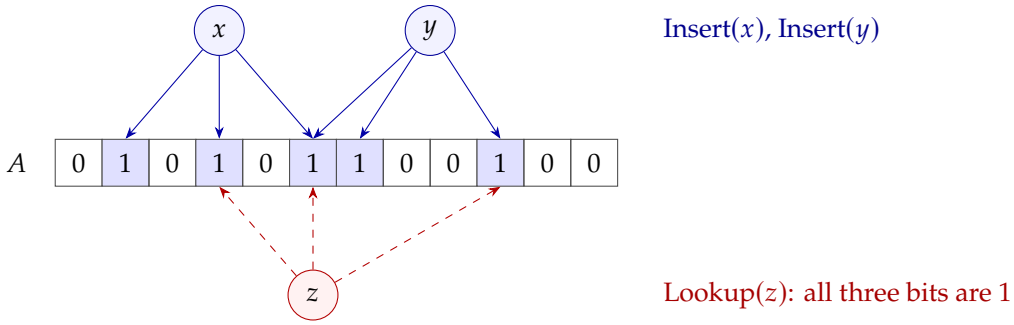


Figure 36: Schematic Bloom filter with $k = 3$. Inserting x and y sets the bits at their hash values. A Lookup of $z \notin S$ whose three bits were all set by these insertions is a false positive.

16.2 Heuristic analysis

There should be a trade-off between space and false positive probability. To quantify it, assume (without justification) that all values $h_i(x)$ are uniformly random and independent, across different i and different x. Insert a data set S into a filter of n bits.

Lemma 16.1 [Probability that a bit is set]. *Under the heuristic assumption, after inserting S, a given bit of A is 1 with probability*

$$1 - \left(1 - \frac{1}{n}\right)^{k|S|} \approx 1 - e^{-k|S|/n} = 1 - e^{-k/b}.$$

Proof. The insertions set $k|S|$ bits, each chosen independently and uniformly. The given bit is missed by each with probability $1 - 1/n$, so it remains 0 with probability $(1 - 1/n)^{k|S|}$. For the approximation, use $1 + z \le e^z$, which is nearly an equality for small $|z|$. With $z = -1/n$,

$$\left(1 - \frac{1}{n}\right)^{k|S|} \le e^{-k|S|/n},$$

and the two sides agree in the limit of large n. □

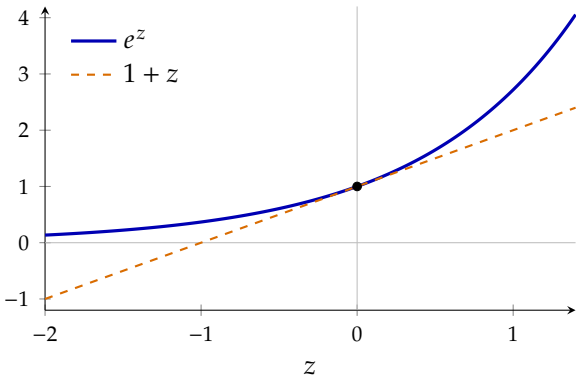


Figure 37: The line $1 + z$ is tangent to e^z at $z = 0$ and lies below it everywhere, so $1 + z \le e^z$ with near-equality for small $|z|$.

Proposition 16.2 [False positive rate]. *Under the heuristic assumption, for $x \notin S$, the false positive probability is approximately*

$$\varepsilon = \left(1 - e^{-k/b}\right)^k,$$

where b is the number of bits per object.

Proof. Lookup(x) is a false positive exactly when all k bits $A[h_1(x)], \ldots, A[h_k(x)]$ are 1. Under the assumption these are k independent uniformly random positions. Treating their values as independent, each is 1 with the probability in Lemma 16.1. □

Proposition 16.3 [Choosing the number of hash functions]. *For fixed b, the error rate ε is minimized by $k \approx (\ln 2)\, b \approx 0.693\, b$. Then*

$$\varepsilon \approx \left(\frac{1}{2}\right)^{(\ln 2)b}, \qquad \textit{equivalently} \qquad b \approx 1.44 \log_2 \frac{1}{\varepsilon}.$$

Proof. Write $p = e^{-k/b}$, so $k = -b \ln p$ and

$$\ln \varepsilon = k \ln(1 - p) = -b \, \ln p \, \ln(1 - p).$$

Minimizing ε means maximizing $\ln p \, \ln(1 - p)$ over $0 < p < 1$. This expression is symmetric under $p \leftrightarrow 1 - p$ and is maximized at $p = 1/2$, that is, at $k = b \ln 2$. Then each bit of x is 1 with probability $1/2$, so $\varepsilon = (1/2)^k = (1/2)^{(\ln 2)b}$. Solving for b gives $b = \log_2(1/\varepsilon)/\ln 2 \approx 1.44 \log_2(1/\varepsilon)$. □

The error rate is exponentially small in the number of bits per object.

Example: With $b = 8$ bits per object, the optimum $k \approx 5.5$, so choose $k = 5$ or $k = 6$. Either gives an error probability of only about 2%.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Bloom Filters.